

Evaluierung von Ansätzen zur Optimierung von Cross Corpus Named Entity Recognition

Studienarbeit

Im Studiengang

Informatik

am Center for Advanced Studies
der Dualen Hochschule Baden-Württemberg

Vorname und Nachname:	Denis Trautner
Matrikelnummer:	5474827
Bearbeitungszeitraum:	14.10.2024 – 13.04.2025
Dualer Partner:	Andreas STIHL AG & Co. KG
Betreuer*in:	M. Sc. Nils Clauß
Prüfer*in:	Dr. rer. nat. Philippe Thomas

Erklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbständig und ohne fremde Hilfe verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Aus fremden Quellen direkt oder indirekt übernommene Gedanken habe ich als solche kenntlich gemacht. Die Arbeit habe ich bisher keinem anderen Prüfungsamt in gleicher oder vergleichbarer Form vorgelegt. Sie wurde bisher auch nicht veröffentlicht.

Ich erkläre weiterhin, dass ich ein unverschlüsseltes digitales Textdokument der Arbeit (in einem der Formate .doc, .docx, .odt, .pdf, .rtf) in Moodle hochgeladen habe.

Schwaikheim, 13.04.2025

(Ort, Datum)

(Unterschrift)

Inhaltsverzeichnis

Abkürzungsverzeichnis	V
Abbildungsverzeichnis.....	VI
Tabellenverzeichnis.....	VII
1 Einleitung	1
1.1 Motivation und Problemstellung	1
1.2 Zielsetzung.....	2
1.3 Abgrenzung.....	3
1.4 Vorgehensweise.....	3
2 Grundlagen	4
2.1 Named Entity Recognition.....	4
2.2 Transformer-Architektur	6
2.3 Evaluationsmetriken.....	10
3 Stand der Forschung.....	12
3.1 Transfer Learning.....	12
3.2 Domain Adaption.....	13
3.3 Adapter Layers.....	13
3.4 Multitask Learning	14
3.5 Datenaugmentierung.....	15
3.6 Ensemble-Methoden	16
3.7 Bewertung der Lösungsansätze	17
4 Methodik	20
4.1 Laborexperiment	20
4.2 CRISP-DM	20
5 Vorbereitung	22
5.1 Auswahl Korpora.....	22
5.2 Analyse/Exploration Korpora.....	24
5.3 Vorverarbeitung.....	26

6	Modellierung	29
6.1	Erstellung Baseline	29
6.2	Adapter Layer Implementierung	30
6.3	Trainings- und Test-Setups	34
7	Evaluation	41
7.1	Baseline	41
7.2	Adapter-Experimente	44
8	Fazit	53
8.1	Kritische Reflexion	54
8.2	Ausblick	55
	Literaturverzeichnis	57

Abkürzungsverzeichnis

BERT	=	Bidirectional encoder representations from transformers
CIRSP-DM	=	Cross-industry standard process for data mining
CRFs	=	Conditional Random Fields
EDA	=	Easy Data Augmentation
GELU	=	Gaussian Error Linear Unit
GPU	=	Graphics Processing Unit
LLM	=	Large Language Model
LSTM	=	Long short-term memory
LoRA	=	Low-Rank Adapter
MMD	=	Maximum Mean Discrepancy
MLM	=	Masked Language Modeling
NLP	=	Natural Language Processing
NER	=	Named Entity Recognition
PLM	=	Pretrained Language Model
ReFT	=	Representation Finetuning
ReLU	=	Rectified Linear Unit

Abbildungsverzeichnis

Abbildung 1: Transformer-Architektur	6
Abbildung 2: Self-Attention in Transformer-Architekturen	9
Abbildung 3: CRISP-DM Prozess	21
Abbildung 4: Klassenverteilung in SETH-Korpus	25
Abbildung 5: Label-Mapping	26
Abbildung 6: Adaptereinbindung und -konfiguration	32
Abbildung 7: Stack-Komposition von Adaptern	35
Abbildung 8: Fusion-Komposition von Adaptern	36
Abbildung 9: Ensemble-Komposition von Adaptern	37
Abbildung 10: SETH-Modell Confusion Matrizen Variome & Amia	43
Abbildung 11: SETH-Modell Confusion Matrix TmVar	44

Tabellenverzeichnis

Tabelle 1: Übersicht über Mutation Corpora	22
Tabelle 2: Übersicht über Cross- & Multi-Lingual Medication Detection Corpora	23
Tabelle 3: Analyse Mutation-Corpora	24
Tabelle 4: Genutzte Adapterkonfiguration	38
Tabelle 5: Baseline-Ergebnisse	41
Tabelle 6: Optimale Trainingsparameter je Adapterkonfiguration	45
Tabelle 7: Ergebnisse Vergleich Adapter/Modelle	45
Tabelle 8: Ergebnis Adapter-Layer-Experimente	47
Tabelle 9: Ergebnisse für grobes vs. granuläres Label-Mapping	51

1 Einleitung

Die automatische Erkennung benannter Entitäten, bekannt als Named Entity Recognition (NER), ist eine zentrale Aufgabe im Bereich des Natural Language Processings (NLP). Die rasante Weiterentwicklung von Deep-Learning-Methoden und vor allem der Einsatz vortrainierter Sprachmodelle wie BERT (Devlin et al., 2019) haben in den letzten Jahren zu bedeutenden Fortschritten geführt. Diese Technologien bilden die Grundlage zahlreicher Anwendungen, von der Informationsbeschaffung bis hin zur Unterstützung komplexer Entscheidungsprozesse in verschiedenen Branchen (Rogers et al., 2020; Sajun et al., 2024). Insbesondere in einer zunehmend digitalisierten Welt gewinnt die Fähigkeit, relevante Informationen aus großen Mengen unstrukturierter Texte effizient zu extrahieren, stetig an Bedeutung. Eine Studie von Gartner prognostiziert, dass bis Ende 2026 mehr als 75% der Unternehmen NLP-Technologien für geschäftskritische Prozesse einsetzen werden (Gartner, 2023).

1.1 Motivation und Problemstellung

Trotz der signifikanten Fortschritte im Bereich der automatischen Erkennung von Entitäten mithilfe neuronaler Netzwerke stoßen NER-Modelle häufig an Grenzen, wenn sie außerhalb ihres ursprünglichen Trainingskontextes eingesetzt werden (Gururangan et al., 2020). Ein Modell, das auf einem gut annotierten und homogenen Korpus exzellente Ergebnisse erzielt, zeigt oftmals drastisch reduzierte Erkennungsgenauigkeiten, sobald es auf andere, wenn auch thematisch verwandte, Datensätze angewendet wird (Lin & Lu, 2018; Sängler et al., 2024). Diese Leistungseinbußen resultieren maßgeblich aus Unterschieden in den statistischen Eigenschaften zwischen dem Trainings- und dem Zielkorpus. Unterschiedliche Schreibstile, variierendes Vokabular, abweichende Satzstrukturen sowie Unterschiede in der Textlänge und im Genre führen dazu, dass die in einem spezifischen Korpus erlernten Muster nur unzureichend auf andere Datensätze übertragbar sind (Z. Liu et al., 2020; Sängler et al., 2024).

Selbst innerhalb derselben Sprache und Domäne können Korpora stark divergieren, etwa hinsichtlich ihres thematischen Fokus, Genres, Stils oder Texttyps. Eine Analyse von Zhang & Zhang (2022) quantifizierte diese Unterschiede anhand lexikalischer Diversität und syntaktischer Komplexität und fand Abweichungen von bis zu 45% zwischen vermeintlich ähnlichen Korpora (Zhang & Zhang, 2022). So unterscheiden sich zum Beispiel journalistische Nachrichtenartikel in Sprache und Aufbau deutlich von informellen Social-Media-Beiträgen oder wissenschaftlichen Fachtexten. Entsprechend variieren auch Wortwahl und Satzstruktur. Ein auf Nachrichtenartikeln trainiertes NER-Modell hat daher häufig Schwierigkeiten, umgangssprachliche Ausdrücke oder spezifischen Jargon in anderen Dokumenttypen adäquat zu erkennen. Dies führt zu Präzisionsverlusten von bis zu 40 Prozentpunkten (Augenstein et al., 2017; Fu et al., 2020).

Zudem tragen unterschiedlich definierte und markierte Annotationen erheblich dazu bei, dass Modelle nicht direkt übertragbar sind (Llorca et al., 2023). Diese Variabilität in der sprachlichen Ausprägung sowie in den Annotationen führt dazu, dass die Leistungsfähigkeit eines NER-Systems stark vom jeweiligen Korpus-Kontext abhängt.

Für die Praxis bedeutet dies eine erhebliche Einschränkung. In realen Anwendungsszenarien stammen die zu verarbeitende Texte selten aus nur einer einzigen Quelle, sondern aus vielfältigen Korpora wie verschiedenen Nachrichtenseiten, Archiven oder Domänentexten. Ein Modell, das auf Korpus A exzellente Ergebnisse liefert, kann auf Korpus B desselben Themengebiets bereits deutlich schwächere Resultate erzielen (Z. Liu et al., 2020; Sängler et al., 2024).

Die gegenwärtige Praxis, für jeden Korpus ein separates Modell zu trainieren und einzusetzen, ist weder ressourceneffizient noch praktikabel für Anwendungen, die kontinuierlich mit neuen und diversen Textquellen konfrontiert werden (Gururangan et al., 2020). Der konventionelle Ansatz des Domain-Finetuning, bei dem ein vortrainiertes Modell auf einen spezifischen Zielkorpus angepasst wird, erzeugt eine unüberschaubare Vielzahl von Modellvarianten – für jede Domäne und jeden Korpus ein separates Modell (Lin & Lu, 2018). Der alternative Versuch, ein einzelnes Modell durch Finetuning gleichzeitig auf mehrere unterschiedliche Korpora anzupassen, führt häufig zu suboptimalen Ergebnissen durch das sogenannte "catastrophic forgetting" (McCloskey & Cohen, 1989), bei dem das Modell Wissen aus früheren Trainingskorpora verliert.

1.2 Zielsetzung

Obwohl die Notwendigkeit des Trainings mit neuen Korpora nicht vollständig eliminiert werden kann, besteht ein dringender Bedarf an einem universellen Modellansatz, der die Effizienz dieses Prozesses maximiert. Statt für jede Domäne separate Modelle zu entwickeln, zielt diese Arbeit darauf ab, ein einzelnes, flexibles Grundmodell zu realisieren, das effizient mit einer begrenzten Anzahl heterogener Korpora umgehen kann. Konkret soll die Leistungsfähigkeit und Generalisierungsfähigkeit von Named Entity Recognition-Modellen in Cross-Corpus-Umgebungen verbessert werden. Dabei sollen verschiedene methodische Ansätze wie Transfer Learning, Domain Adaptation, Multitask Learning und Datenaugmentation bewertet und die vermeintlich vielversprechendste Methode prototypisch implementiert werden.

Daraus ergibt sich die zentrale Forschungsfrage: Wie können die Leistungsfähigkeit und Generalisierungsfähigkeit von Named Entity Recognition-Modellen in Cross-Corpus-Umgebungen verbessert werden, sodass ein universelles Modell entsteht, das trotz heterogener Textquellen konsistent hohe Erkennungsleistungen erzielt?

1.3 Abgrenzung

Um den Umfang der Arbeit klar zu begrenzen, werden bestimmte Aspekte ausgeschlossen. Erstens konzentriert sich die Arbeit ausschließlich auf Textkorpora derselben Sprachen. Andere Sprachen und damit verbundene zusätzliche Herausforderungen (z. B. unterschiedliche Grammatik oder Schrift) bleiben unberücksichtigt. Zweitens betrachtet die Arbeit nur Korpora innerhalb derselben Domäne. Es wird ausschließlich innerhalb einer einheitlichen Domäne gearbeitet, um fachspezifische Unterschiede weitgehend zu minimieren. Variationen in den Annotationen, wie unterschiedliche Definitionen von Entitätstypen und variierende Markierungspraktiken, werden als gegebene Rahmenbedingungen akzeptiert, ohne dass deren Harmonisierung im Fokus der Arbeit steht. Zudem wird aufgrund des begrenzten Umfangs dieser Arbeit auf eine normale Literaturrecherche zurückgegriffen, anstatt ein umfassendes Systematic Literature Review durchzuführen. Dies erlaubt einen praxisorientierten Ansatz, bei dem der methodische Schwerpunkt auf der Optimierung und Generalisierung von NER-Modellen innerhalb eines konsistenten sprachlichen und thematischen Kontextes liegt, während interlinguale oder domänenübergreifende Herausforderungen explizit ausgeklammert werden.

1.4 Vorgehensweise

Die Arbeit gliedert sich in acht Kapitel. In Kapitel 2 werden die theoretischen Grundlagen zu Named Entity Recognition, Transformer-Modellen und relevanten Evaluationsmetriken erläutert. Anschließend gibt Kapitel 3 einen komprimierten Überblick über den aktuellen Forschungsstand, wobei potenzielle Lösungsansätze für das Cross-Corpus-Problem vorgestellt werden. In Kapitel 3.7 wird dabei der vielversprechendste Ansatz ausgewählt, für den anschließend eine prototypische Implementierung in den nächsten Kapiteln erfolgt. Kapitel 4 beschreibt die methodische Vorgehensweise der Implementierung und Evaluation im Detail. Kapitel 5 widmet sich den praktischen Vorbereitungen, einschließlich der Auswahl, Analyse und Vorverarbeitung der Textkorpora. Die eigentliche Implementierung des ausgewählten Lösungsverfahrens wird in Kapitel 6 durchgeführt. In Kapitel 7 werden die experimentellen Ergebnisse präsentiert und kritisch diskutiert, bevor Kapitel 8 die zentralen Erkenntnisse zusammenfasst, Limitationen erörtert und einen Ausblick auf zukünftige Forschungsperspektiven gibt.

2 Grundlagen

In diesem Kapitel werden die Grundlagen in Bezug auf Named Entity Recognition, Transformer Architekturen und relevante Evaluationsmetriken dargelegt.

2.1 Named Entity Recognition

Named Entity Recognition (NER) bezeichnet die automatische Identifikation und Klassifikation benannter Entitäten in Texten, wie etwa Personennamen, geografischen Orten, Organisationen, Zeitangaben oder fachspezifischen Begriffen. Als zentrales Verfahren der Informationsextraktion ist NER eine grundlegende Komponente vieler Anwendungen im Bereich des Natural Language Processing (NLP), darunter Fragebeantwortungssysteme, semantische Suche, Textzusammenfassung und Relationserkennung (Jurafsky & Martin, 2013; Rogers et al., 2020).

NER wird als Sequenzklassifikationsproblem modelliert, bei dem ein Text zunächst in sogenannte Tokens zerlegt wird. Tokens sind die kleinsten bedeutungstragenden Einheiten eines Textes und bestehen meist aus einzelnen Wörtern, Satzzeichen oder Subworteinheiten (Nadeau & Sekine, 2009). Ziel eines NER-Systems ist es, zusammenhängende Tokenfolgen – beispielsweise „Donald Trump“ oder „Deutsche Bank“ – als Entitäten zu erkennen und ihnen einen semantischen Typ (z. B. PERSON, ORGANIZATION, LOCATION) zuzuweisen.

Um mehrteilige Entitäten korrekt zu erfassen, wird häufig das sogenannte IOB-Tagging-Schema verwendet, das auch als BIO-Schema bekannt ist. Dabei steht B für den Beginn (Beginning) einer Entität, I für ein fortgesetztes (Inside) Token derselben Entität und O für Tokens außerhalb einer Entität (Ramshaw & Marcus, 1999). So wird beispielsweise die Entität „Deutsche Bank“ mit Deutsche/B-ORG und Bank/I-ORG markiert.

Die Entwicklung von NER-Systemen lässt sich historisch in drei technologische Phasen unterteilen (Devlin et al., 2019; Lample et al., 2016; Nadeau & Sekine, 2009). Erste Systeme setzten auf regelbasierten Verfahren, bei denen manuell definierte linguistische Muster, reguläre Ausdrücke und vordefinierte Listen (Gazetteers) verwendet wurden, um Entitäten zu identifizieren (Appelt et al., 1993; Wacholder et al., 1997). Diese Ansätze waren zwar präzise in eng abgegrenzten Domänen, jedoch kaum skalierbar und extrem anfällig gegenüber sprachlicher Varianz. Ab den 2000er Jahren wurden regelbasierte Verfahren zunehmend durch statistische Modelle wie Hidden Markov Models, Maximum Entropy Markov Models und insbesondere Conditional Random Fields (CRFs) abgelöst (Lafferty et al., 2001).

CRFs sind probabilistische Sequenzmodelle, die es ermöglichen, Abhängigkeiten zwischen benachbarten Labels zu berücksichtigen. So lernt das Model unter Anderem, dass auf ein B-LOC mit hoher Wahrscheinlichkeit ein I-LOC folgt (Lafferty et al., 2001).

In Kombination mit Feature-Engineering, etwa zu Wortformen, Groß-/Kleinschreibung oder Kontextfenstern, führten diese Modelle zu einer deutlich verbesserten Performanz auf standardisierten NER-Benchmarks wie dem CoNLL-2003-Datensatz (Tjong Kim Sang & Meulder, 2003).

Mit dem Aufkommen tiefer neuronaler Netze veränderte sich der Ansatz zur Lösung der NER-Aufgabe nochmal fundamental. Zunächst dominierten rekurrente neuronale Netzwerke wie Long Short-Term Memory-Netze (LSTMs), die durch ihre Fähigkeit, sequentielle Abhängigkeiten zu erfassen, in Kombination mit CRFs besonders effektiv eingesetzt werden konnten (Hochreiter & Schmidhuber, 1997; Huang et al., 2015; Lample et al., 2016). Einen entscheidenden Wendepunkt markierte jedoch die Einführung des Transformer-Modells (Vaswani et al., 2017). Dessen zentrale Neuerung war der sogenannte Self-Attention-Mechanismus, der es jedem Token einer Eingabesequenz erlaubt, in Relation zu allen anderen Tokens gewichtet betrachtet zu werden. Dadurch wurde es möglich, auch langfristige Abhängigkeiten über den gesamten Text hinweg effizient zu modellieren. Bei RNNs war nur eine sequentielle Verarbeitung möglich. Bei Transformer Architektur ist der Lernprozess zudem vollständig parallelisierbar und damit auch ökonomisch effizienter (Vaswani et al., 2017).

Aufbauend auf dieser Architektur wurde 2018 mit BERT (Bidirectional Encoder Representations from Transformers) ein Sprachmodell vorgestellt, das einen fundamentalen Fortschritt für viele NLP-Aufgaben bedeutete, darunter auch NER (Devlin et al., 2019). BERT ist ein vortrainiertes Sprachmodell, das zunächst auf großen Mengen unannotierter Textdaten mittels sogenanntem Masked Language Modeling trainiert wird. Dabei werden einzelne Tokens im Satz zufällig maskiert, und das Modell muss lernen, diese anhand des umgebenden Kontexts zu rekonstruieren. Eine Besonderheit von BERT ist, dass es dabei den Kontext bidirektional berücksichtigt. Dies steht im Gegensatz zu früheren Modellen, die nur den vorangehenden oder nachfolgenden Kontext einbezogen haben. Dadurch kommt es zu einer tieferen semantischen Einbettung der Tokens. Durch ein anschließendes Finetuning kann BERT effizient auf spezifische Aufgaben wie NER angepasst werden, oft mit signifikanten Leistungssteigerungen, auch bei begrenzter Menge an Trainingsdaten (Devlin et al., 2019).

BERT entwickelte sich rasch zum Quasi-Standard in vielen NLP-Aufgaben und wurde Ausgangspunkt zahlreicher Weiterentwicklungen wie RoBERTa (L. Zhuang et al., 2021), ALBERT (Lan et al., 2019) oder DistilBERT (Sanh et al., 2019), die jeweils unterschiedliche Aspekte wie Trainingsdauer, Modellgröße oder Datenrobustheit optimieren. Die Prinzipien der Transformer-Architektur, wie sie BERT verkörpert, werden im Kapitel 2.2 näher erläutert.

Trotz dieser Fortschritte bleibt NER in realweltlichen Anwendungsszenarien, insbesondere in heterogenen oder domänenübergreifenden Umgebungen, eine Herausforderung. Modelle, die auf spezifischen Korpora trainiert wurden, generalisieren oft schlecht auf andere Textsorten

wie Social Media, juristische Texte oder wissenschaftliche Abstracts (Augenstein et al., 2017; Z. Liu et al., 2020). Hinzu kommen Unterschiede in den Annotationen zwischen Korpora, etwa bei der Definition und Granularität von Entitätstypen, die eine direkte Übertragbarkeit weiter erschweren (Alshammari & Alanazi, 2021; Llorca et al., 2023). Named Entity Recognition bleibt somit trotz zugenommener technischer Reife ein aktives Forschungsfeld mit Bedarf an robusten, domänenübergreifenden Lösungen.

2.2 Transformer-Architektur

Transformer-Modelle bilden die Grundlage vieler moderner Sprachmodelle. Diese sogenannten Pretrained Language Models (PLMs) werden auf großen Mengen unannotierter Textdaten vortrainiert und anschließend für spezifische Aufgaben wie NER, Textklassifikation oder Fragebeantwortung feinjustiert. Das Pretraining dient dabei dem Erwerb eines allgemeinen Sprachverständnisses, das über viele Aufgaben hinweg nutzbar gemacht werden kann (Rogers et al., 2020).

Die Architektur des ursprünglichen Transformer-Modells besteht aus zwei Hauptkomponenten: dem Encoder, der die Eingabesequenz in eine dichte, kontextualisierte Repräsentation überführt, und dem Decoder, der auf Basis dieser Repräsentation eine Ausgabesequenz generiert (Vaswani et al., 2017). Abbildung 1 zeigt diese ursprünglich vorgestellte Transformer Architektur. Sie verwendet N parallele Encoder- (links) und N parallele Decoderblöcke (rechts).

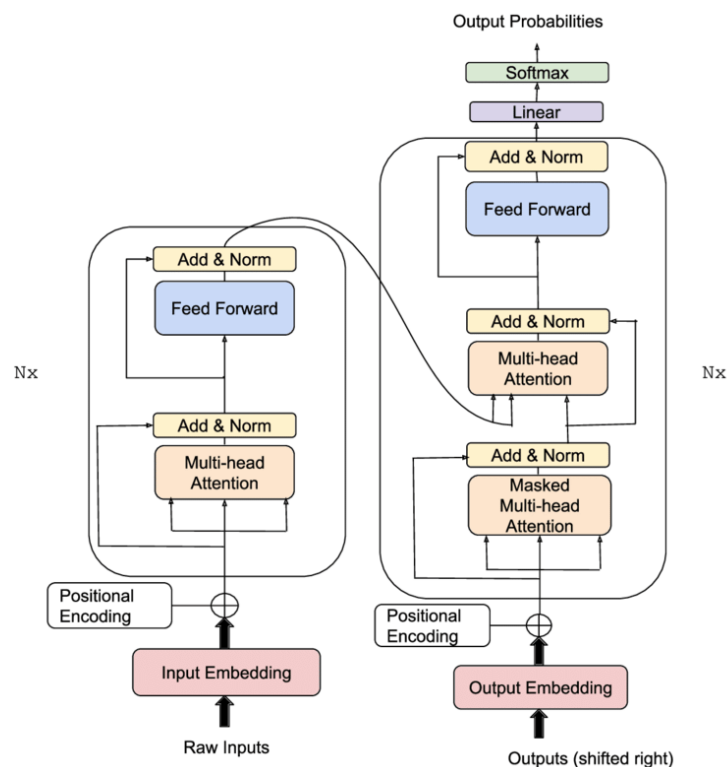


Abbildung 1: Transformer-Architektur mit Encoder-Decoder-Struktur basierend auf Vaswani et al. (2017). Der Decoder verarbeitet die Eingabesequenz und erzeugt eine Repräsentation, die der Decoder mithilfe von Attention-Mechanismen nutzt, um eine Ausgabesequenz zu generieren. Bild aus Gaser (2022)

Der Encoder erhält als Eingabe eine tokenisierte Textsequenz, deren einzelne Elemente in einem ersten Schritt in kontinuierliche Vektoren überführt werden. Diese Vektoren werden anschließend durch mehrere Schichten von Self-Attention und nichtlinearen Transformationen weiterverarbeitet. So entsteht an jeder Position eine Repräsentation, die nicht nur die Information des jeweiligen Tokens enthält, sondern auch dessen Beziehung zu allen anderen Tokens in der Sequenz berücksichtigt. Der Encoder erzeugt somit eine Reihe kontextsensitiver Repräsentationen, die semantische und syntaktische Strukturen innerhalb des Textes erfassen (Goodfellow et al., 2016; Vaswani et al., 2017).

Der Decoder nutzt diese Encoder-Ausgaben als zusätzliche Eingabe, um darauf basierend eine neue Sequenz zu generieren. Die Ausgabe erfolgt autoregressiv, das heißt, jedes Token wird schrittweise erzeugt und basiert auf den zuvor generierten Tokens sowie auf den Informationen aus dem Encoder. Der Decoder verfügt neben einem eigenen Self-Attention-Mechanismus auch über eine Cross-Attention-Komponente, die es ihm ermöglicht, gezielt auf relevante Teile der Encoder-Ausgaben zuzugreifen. Diese Architektur macht den Decoder besonders geeignet für Aufgaben, bei denen eine sprachlich sinnvolle Ausgabe auf Basis einer vorliegenden Eingabe entstehen soll (Vaswani et al., 2017).

Diese Trennung zwischen Encoder und Decoder erlaubt eine gezielte Anpassung der Architektur an verschiedene Anwendungsfälle. Für viele Aufgaben des Sprachverstehens wie Klassifikation, Sentimentanalyse oder NER ist keine Generierung neuer Textsequenzen erforderlich (Rogers et al., 2020). Stattdessen stehen die Analyse und semantische Interpretation eines bestehenden Textes im Vordergrund. In solchen Fällen werden Encoder-only-Modelle eingesetzt, die auf die kontextuelle Verarbeitung der Eingabesequenz spezialisiert sind. Ein prominentes Beispiel für ein solches Modell ist BERT (Bidirectional Encoder Representations from Transformers), das vollständig auf der Encoder-Komponente des ursprünglichen Transformer-Modells basiert (Devlin et al., 2019; Vaswani et al., 2017). BERT wurde mit dem Ziel entwickelt, tiefe bidirektionale Repräsentationen zu lernen, bei denen jedes Token im Kontext seiner gesamten Umgebung interpretiert wird. Diese Eigenschaft macht es besonders geeignet für sequenzielle Klassifikationsaufgaben wie Named Entity Recognition. Da sich die vorliegende Arbeit auf genau diese Aufgabe konzentriert, liegt der Fokus im Folgenden auf der näheren Beschreibung der Encoder-Architektur am Beispiel von BERT.

Encoder-Architektur im Detail

Wie in Abbildung 1 bereits aufgezeigt, besteht ein Encoder aus mehreren identisch aufgebauten Schichten, sogenannten Encoder Blöcken. In der Basisvariante des BERT-Modells sind es beispielsweise zwölf solcher Blöcke, die sequenziell angeordnet sind (Rogers et al., 2020).

Jeder Block verfolgt dasselbe Grundprinzip: Er verarbeitet eine Sequenz von Token-Repräsentationen, verbessert deren Kontextbezug und leitet die angereicherten Repräsentationen an die nächste Schicht weiter. Die Architektur eines solchen Blocks umfasst drei zentrale Komponenten: den Self-Attention-Mechanismus, ein Feedforward-Netzwerk sowie Residualverbindungen und Layer Normalization zur Stabilisierung des Trainings (Vaswani et al., 2017).

Im Zentrum steht der Self-Attention-Mechanismus, der es dem Modell ermöglicht, für jedes Token einer Eingabesequenz zu bestimmen, welche anderen Tokens für dessen Bedeutung relevant sind. Dabei wird jedes Token einer Sequenz durch drei Vektoren dargestellt: Query, Key und Value. Die Attention beziehungsweise Abhängigkeit zwischen zwei Tokens wird über das Skalarprodukt zwischen dem Query-Vektor des aktuellen Tokens und den Key-Vektoren aller anderen Tokens in der Sequenz bestimmt. Anschließend wird durch die Softmax-Funktion eine Normalisierung durchgeführt. Daraus entstehen Wahrscheinlichkeitswerte, die aussagen, wie viel Aufmerksamkeit ein Token einem anderen Token innerhalb der gleichen Sequenz widmen soll. Der Value-Vektor wird gemäß den Aufmerksamkeitswerten gewichtet. Dies nennt sich Scaled Dot-Product Attention und kann durch folgende Formel dargestellt werden (Sajun et al., 2024):

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Hierbei sind:

- Q: Querymatrix, welche die Abfrageinformationen enthält (Dimension: $n \times d_k$)
- K: Key-Matrix, die die Schlüsselrepräsentationen für jedes Eingabeelement enthält (Dimension: $n \times d_k$)
- V: Matrix der Werte, die die tatsächlichen Informationen enthalten, die an die nächste Schicht weitergegeben werden (Dimension: $n \times d_v$)
- n: Anzahl der Token in einer Sequenz
- d_k : Dimension der Query- und Key Vektoren
- d_v : Dimension der Value-Vektoren

Multi-Head-Attention beschreibt den Mechanismus, dass die Attention aus verschiedenen Perspektiven berechnet wird. In Abbildung 2 wird deutlich wie die eben beschriebene Funktion mehrfach parallel berechnet und anschließend konkateniert und linear zusammengeführt wird (Rogers et al., 2020; Sajun et al., 2024).

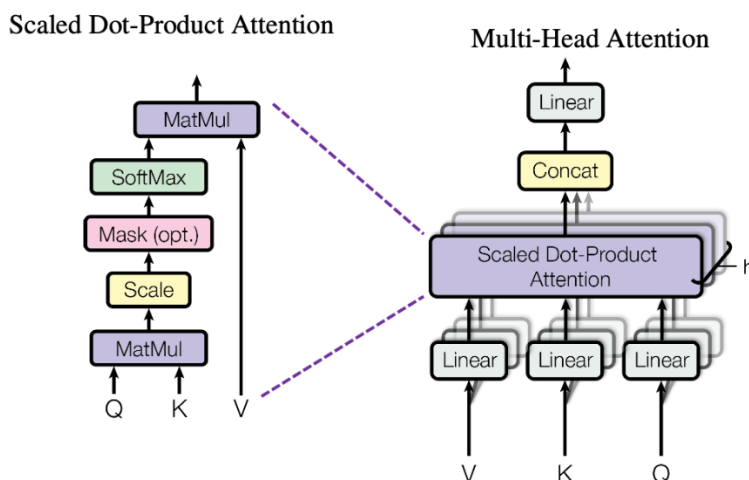


Abbildung 2: Self-Attention in Transformer-Architekturen- Darstellung des Multi-Head Self-Attention-Mechanismus, bei dem mehrere parallele Self-Attention-Blöcke kontextabhängige Repräsentationen einer Sequenz erzeugen und anschließend zusammengeführt werden. Konzept und Abbildung aus Vaswani et al.(2017)

Jeder Head besitzt eigene Projektionsmatrizen für Q, K und V und fokussiert sich auf unterschiedliche Aspekte der Tokenbeziehungen. Dieses Prinzip wird als Multi-Head Attention bezeichnet und erlaubt es dem Modell, vielfältige semantische Muster gleichzeitig zu erfassen.

Nach der Berechnung der Self-Attention wird die Ausgabe durch ein Feed-Forward Netzwerk verarbeitet, welches eine nicht-lineare Transformation der Daten vornimmt. Dies ermöglicht es, komplexere Muster in den Daten zu lernen, die über reine Beziehungen zwischen den Tokens hinausgehen (Vaswani et al., 2017). Wie bereits in Abbildung 1 dargestellt, gibt es nach dem Self Attention Mechanismus als auch dem Feed-Forward Netzwerk ein "Add & Norm"-Block. Add ist eine Residual Connection, die dafür sorgt, dass die Eingabe der Schicht unverändert zur Ausgabe der Schicht addiert wird. Diese Residual-Connections verhindern den Verlust von wichtigen Informationen und helfen, das Problem der „Vanishing Gradients“ zu beheben, das bei tiefen neuronalen Netzwerken auftreten kann. Es bezieht sich auf die Situation, in der die Gradienten der Verlustfunktion gegenüber den Gewichten der vorherigen Schichten während des Backpropagation Prozesses sehr klein werden (Lecun et al., 1998).

Training von Encoder-Schichten

Die wiederholte Anwendung von Self-Attention, Feedforward-Netzwerken, Residualverbindungen und Layer-Normalisierung in mehreren gestapelten Encoder Schichten ermöglicht es dem Modell, mit jeder Ebene ein tieferes und abstrakteres Verständnis der Eingabesequenz zu entwickeln. Doch erst durch ein entsprechendes Trainingsverfahren wird diese Architektur mit sprachlichem Wissen angereichert (Rogers et al., 2020).

Dafür werden die Eingabedaten zunächst tokenisiert und durch drei Arten von Embeddings dargestellt: Token Embeddings kodieren die Bedeutung einzelner Tokens, Segment Embeddings markieren unterschiedliche Satzteile wie etwa Satz A und B, und Positional Embeddings liefern Informationen zur Reihenfolge der Tokens, da der Transformer selbst keine Positionsinformation berücksichtigt.

Das Training von Encodern erfolgt typischerweise in zwei Phasen. In der Pretraining-Phase wird das Modell auf großen, unannotierten Textkorpora mithilfe zweier Aufgaben trainiert. Beim Masked Language Modeling (MLM) werden ca. 15 % der Tokens maskiert, und das Modell muss sie aus dem Kontext heraus vorhersagen. Die zweite Aufgabe, Next Sentence Prediction (NSP), besteht darin, zu entscheiden, ob zwei gegebene Sätze inhaltlich zusammengehören. Beide Aufgaben fördern das Erlernen eines allgemeinen, kontextsensitiven Sprachverständnisses (Devlin et al., 2019).

In der anschließenden Finetuning-Phase wird das vortrainierte Modell an eine konkrete Zielaufgabe wie Named Entity Recognition angepasst. Dazu wird typischerweise eine einfache Klassifikationsschicht ergänzt und das gesamte Modell auf einem kleineren, spezifischen Datensatz weitertrainiert. Da das Modell bereits allgemeine Sprachmuster verinnerlicht hat, genügt oft eine begrenzte Menge annotierter Daten für leistungsfähige Ergebnisse (Devlin et al., 2019; Sanh et al., 2019).

2.3 Evaluationsmetriken

Nachdem im vorherigen Kapitel erklärt wurde, wie der Trainingsprozess abläuft, soll nun erläutert werden mit welchen Metriken die Leistungsfähigkeit eines Modells bewertet werden kann. Für NER-Modellen werden üblicherweise die Metriken Precision, Recall und F1-Score verwendet. Während Precision den Anteil korrekt erkannter Entitäten unter allen Vorhersagen beschreibt, misst Recall, wie viele der tatsächlichen Entitäten korrekt erkannt wurden. Der F1-Score bildet das harmonische Mittel dieser beiden Größen (Jiang et al., 2016).

Für die Bewertung von NER-Modellen ist entscheidend, auf welcher Ebene die Evaluation durchgeführt wird. Bei der Bewertung auf Tokenebene wird jedes einzelne Token berücksichtigt, selbst wenn es nur Teil einer mehrteiligen Entität ist. Die strengere Bewertung auf Entitätsebene hingegen erkennt eine Vorhersage nur dann als korrekt an, wenn die vollständige Entität exakt und mit dem richtigen Typ identifiziert wurde (Tjong Kim Sang & Meulder, 2003).

Darüber hinaus unterscheidet man bei der Aggregation der Ergebnisse zwischen micro- und macro-averaging. Beim Micro Averaging werden alle Vorhersagen unabhängig von ihrem Typ gemeinsam ausgewertet. Das bedeutet, dass zunächst die Gesamtanzahl korrekter, falscher und fehlender Vorhersagen über alle Klassen hinweg gezählt wird.

Anschließend werden auf dieser Basis die Metriken berechnet. Häufig vorkommende Klassen haben dadurch einen größeren Einfluss auf das Ergebnis. Beim Macro Averaging hingegen wird für jede Klasse einzeln ein Precision-, Recall- und F1-Wert berechnet. Diese Klassenergebnisse werden anschließend gleichgewichtet gemittelt. So erhält jede Klasse unabhängig von ihrer Häufigkeit den gleichen Einfluss auf die Gesamtbewertung (Takahashi et al., 2022; Tjong Kim Sang & Meulder, 2003).

Die sonst häufig verwendete Metrik Accuracy, welchen den Anteil der korrekt gelabelten Tokens an der Gesamtanzahl aller Tokens misst, ist bei NER-Modellen nur eingeschränkt aussagekräftig. Dies liegt daran, dass ein Großteil der Tokens typischerweise zur „O“-Klasse gehören, welche keine Entität darstellt. Ein Modell kann deshalb eine hohe Accuracy erreichen, selbst wenn es relevante Entitäten kaum oder gar nicht erkennt (Tsai et al., 2006). Die alleinige Betrachtung der Accuracy führt daher zu einer verzerrten Einschätzung der Modellleistung. Stattdessen hat sich ein micro-averaged F1-Score auf Entitätsebene als Standard für die Evaluation von NER-Modellen etabliert.

3 Stand der Forschung

Zur Verbesserung der Generalisierbarkeit im Kontext des beschriebenen Cross-Corpus-Problems kommen eine Reihe methodischer Ansätze in Betracht, die im Folgenden beschrieben werden. In Abschnitt 3.7 erfolgt anschließend eine Bewertung, um die vielversprechendste Methode zu identifizieren, die in den darauffolgenden Kapiteln implementiert werden soll.

3.1 Transfer Learning

Ein Ansatz zur Bewältigung des Cross-Corpus-Problems stellt das Transfer Learning dar. Dieser Ansatz basiert auf der Idee, ein Sprachmodell zunächst auf einer großen, allgemeinen Textbasis vorzutrainieren, um ein tiefes und vielseitiges Verständnis sprachlicher Strukturen, semantischer Zusammenhänge und syntaktischer Muster zu erlernen (Devlin et al., 2019). Dieses vortrainierte Modell dient anschließend als Ausgangspunkt für eine domänenspezifische Anpassung. Im Gegensatz zum klassischen Fine-Tuning auf einem einzelnen Zielkorpus, das wie in der Einleitung beschrieben häufig zu einer schlechten Übertragbarkeit auf andere, thematisch verwandte Korpora führt, ermöglicht Transfer Learning das gleichzeitige Training mit mehreren heterogenen Korpora derselben Domäne. Dabei lernt das Modell abstrakte, korpusübergreifende Repräsentationen, anstatt sich auf korpuspezifische Eigenheiten wie bestimmte Ausdrücke oder Textformen zu verlassen (F. Zhuang et al., 2021).

Hierzu gibt es einige vielversprechende Ansätze in der aktuellen Forschung, die ähnliche Zielsetzungen verfolgen. So demonstrieren Gururangan et al. (2020), dass ein zusätzliches domänenspezifisches Vortraining auf thematisch verwandten, aber formal diversen Texten die Generalisierungsfähigkeit von Sprachmodellen für Klassifikationsaufgaben verbessert. Die Ergebnisse einer Studie von (Z. Liu et al., 2020), in der ein CrossNER-Benchmark-Setup entwickelt wurde, deuten darauf hin, dass ein gemeinsames Training auf mehreren Korpora zu erheblichen Leistungssteigerungen führt, wenn das Modell auf bisher ungesehene Quellen angewendet wird.

Auch im Bereich der Sentimentanalyse und anderen Klassifikationsaufgaben zeigen Howard & Ruder (2018), dass ein Transfer-Learning-Modell, das auf einer Vielzahl von Texten aus unterschiedlichen Quellen trainiert wurde, deutlich besser in der Lage ist, robuste Klassifikationen in neuen Textsorten zu erstellen. Hier wurde auch gezeigt, dass dies auch der Fall ist, wenn sich diese Texte in Struktur, Länge oder Vokabular stark unterscheiden.

Ergänzend hierzu demonstrieren Khashabi et al. (2020) im Kontext von Fragebeantwortungssystemen, dass ein Modell, das auf heterogenen QA-Datensätzen trainiert wurde, flexibel auf verschiedene Fragestellungsformate reagieren kann.

Dieser Ansatz führt zu einem System, das nicht nur die spezifischen Eigenschaften einzelner Korpora übernimmt, sondern ein umfassendes und generalisierendes Sprachverständnis entwickelt, das sich auf diverse Anwendungsfälle übertragen lässt (Khashabi et al., 2020).

Zusammengefasst verdeutlichen diese Ansätze, dass Transfer Learning, insbesondere in Form eines kombinierten Trainings auf mehreren Korpora, einen effektiven Weg darstellt, um Modelle zu entwickeln, die korpusübergreifende, abstrahierte Repräsentationen erlernen.

3.2 Domain Adaption

Domain Adaptation ist eine Methode, die systematische Unterschiede zwischen verschiedenen Datensätzen innerhalb einer gemeinsamen Domäne gezielt ausgleicht. Während der allgemeine Transfer-Learning-Ansatz darauf abzielt, durch das Vortraining auf einer umfangreichen Textbasis und anschließendes Finetuning auf einer spezifischen Zielaufgabe abstrakte, korpusübergreifende Repräsentationen zu erlernen (Devlin et al., 2019; Gururangan et al., 2020), geht Domain Adaptation einen Schritt weiter. Sie ergänzt diese Grundidee, indem sie zusätzliche Trainingsmechanismen einsetzt, um die statistischen Unterschiede zwischen den einzelnen Quellen aktiv zu minimieren (C. Chu & Wang, 2018).

Dabei wird nicht nur die Anpassung an die Zielaufgabe optimiert, sondern es wird auch darauf geachtet, dass die extrahierten Merkmale domänenunspezifisch sind. Adversariale Trainingsverfahren, wie sie beispielsweise von Ganin et al. (2017) entwickelt wurden, integrieren einen Domänenklassifikator über einen Gradient Reversal Layer, sodass das Modell lernt, Merkmale zu extrahieren, die keine Informationen über die Domänenzugehörigkeit enthalten. Ergänzend dazu kommen Methoden zum Einsatz, die direkt die Divergenz zwischen den Feature-Verteilungen minimieren. Beispiele für diese Methoden sind das Maximum Mean Discrepancy (MMD) oder die Minimierung des Classifier Discrepancy (T. Chu et al., 2022; Saito et al., 2017).

Obwohl Domain Adaptation als eigenständige Methode konzipiert ist, baut sie auf den Prinzipien des Transfer Learning auf. Sie profitiert von bereits erlernten allgemeinen Sprachrepräsentationen und erweitert diese um Mechanismen zum Ausgleich interner Unterschiede (C. Chu & Wang, 2018). Während Domain Adaptation in der Forschung häufig zur Angleichung an eine neue Domäne verwendet wird, könnten die zugrunde liegenden Mechanismen in der vorliegenden Problemstellung dazu beitragen, die Generalisierung zwischen verschiedenen Korpora innerhalb derselben Domäne zu verbessern.

3.3 Adapter Layers

Adapter Layers sind eine parameter-effiziente Methode zur Anpassung vortrainierter Sprachmodelle an neue Aufgaben oder Datenquellen.

Grundsätzlich können Adapter Layers als eine Untermethode des Transfer Learning betrachtet werden (Bapna & Firat, 2019). Der Kernansatz besteht darin, dass anstelle einer vollständigen Anpassung des gesamten Modells während des Finetunings lediglich kleine, zusätzliche Schichten in das Modell eingefügt werden. Diese Schichten bezeichnet man als Adapter. Die Hauptgewichte des vortrainierten Modells bleiben dabei weitgehend unverändert, wodurch das bereits erworbene allgemeine Sprachverständnis erhalten bleibt und das Risiko des „catastrophic forgetting“ reduziert wird (Pfeiffer, Rücklé et al., 2020).

Das grundlegende Konzept von Adapter Layers besteht darin, die Anpassung an spezifische Aufgaben oder Korpora von den allgemeinen, vortrainierten Sprachrepräsentationen zu entkoppeln. Houlsby et al. (2019) zeigen in ihrer Arbeit, dass der Einsatz von Adapter Layers in verschiedenen NLP-Aufgaben vergleichbare Ergebnisse wie herkömmliches Finetuning erzielen kann, während nur ein verhältnismäßig kleiner Anteil an Parametern trainiert werden muss. Der modulare Ansatz des Konzepts erlaubt es, für unterschiedliche Datensätze oder Sprachen jeweils spezifische Adapter zu trainieren, ohne separate vollständige Modelle verwalten zu müssen. Pfeiffer, Kamath et al. (2020) erweiterten diesen Ansatz mit AdapterFusion, einer Methode, die es ermöglicht, Informationen aus mehreren Adapter-Modulen zu kombinieren und so die Generalisierungsfähigkeit weiter zu verbessern.

Darüber hinaus werden Adapter Layers auch erfolgreich in Cross-Lingual-Szenarien eingesetzt. Das MAD-X-Framework von Pfeiffer, Vulić et al. (2020) demonstriert, dass durch den Einsatz sprachspezifischer Adapter robustes Wissen zwischen verschiedenen Sprachen übertragen werden kann. Dies kann vor allem in ressourcenarmen Sprachen beziehungsweise Trainingsdatensätzen von Vorteil sein. Ebenso nutzen Bapna & Firat (2019) in der neuronalen maschinellen Übersetzung Residual-Adapter, die in Transformer-Modelle integriert sind, um eine effiziente Anpassung über verschiedene Sprachpaare hinweg zu ermöglichen.

Die in Cross-Lingual-Anwendungen erzielten Ergebnisse bieten dabei auch interessante Perspektiven für das vorliegende Cross-Corpus-Problem. Beide Szenarien teilen die Herausforderung, mit heterogenen Daten umzugehen, sei es in Form unterschiedlicher Sprachen oder variierender Schreibstile, Textlängen und Annotationen innerhalb derselben Domäne.

3.4 Multitask Learning

Beim Multitask-Learning entsteht ein Modell das gleichzeitig auf mehreren Aufgaben trainiert wird, um gemeinsame, robuste Repräsentationen zu erlernen. Im Kontext von Cross-Corpus-Problemen, bei denen verschiedene Korpora innerhalb derselben Domäne unterschiedliche Schreibstile, Längen und Annotationen aufweisen, kann man jeden Datensatz als eine verwandte, aber leicht unterschiedliche Aufgabe betrachten. Dadurch werden beim Training gemeinsame als auch spezifische Merkmale erfasst (X. Liu et al., 2019; Ruder, 2017).

Multitask-Learning-Systeme basieren in der Regel auf einem gemeinsamen vortrainierten Encoder-Modell wie BERT und erstellen dann Aufgabenspezifische Head, die für die jeweilige Vorhersage zuständig sind. Ein solcher Head ist in der Regel eine zusätzliche Feedforward-Schicht, die an den gemeinsamen Encoder angeschlossen wird. Diese task-spezifischen Heads sind nicht mit den internen Attention Heads der Transformer-Architektur zu verwechseln, die fest in BERT integriert sind. Stattdessen wandeln sie die vom Encoder erlernten allgemeinen Repräsentationen in konkrete Outputs für die jeweilige Aufgabe um (Howard & Ruder, 2018).

Während der gemeinsame Encoder die fundamentalen sprachlichen Merkmale aller Datensätze lernt, ermöglichen die spezifischen Heads eine differenzierte Berücksichtigung korpus- bzw. aufgabenspezifische Nuancen, ohne dass das Modell in seiner Gesamtstruktur zu stark auf einen einzelnen Datensatz fokussiert (X. Liu et al., 2019; Ruder, 2017).

Erste Erkenntnisse zur Wirksamkeit des Multitask Learning stammen aus der Arbeit von Caruana (1997), der zeigte, dass Modelle, die auf mehreren Aufgaben gleichzeitig trainiert werden, oft besser generalisieren als solche, die nur auf eine einzige Aufgabe optimiert werden. Auch neuere Arbeiten, wie die von X. Liu et al. (2019) bestätigen diese Erkenntnisse und zeigen, dass das gleichzeitige Training auf verschiedenen Aufgaben robuste gemeinsame Repräsentationen fördert.

Zusammengefasst ermöglicht Multitask Learning in Cross-Corpus-Problemen, dass ein gemeinsamer Encoder die fundamentalen sprachlichen Merkmale aller Korpora erfasst und die task-spezifischen Heads dann die feinen Unterschiede der einzelnen Datensätze modellieren. So entsteht ein Modell, das die Heterogenität verschiedener Datensätze effektiv integriert und konsistente, generalisierende Vorhersagen liefert.

3.5 Datenaugmentierung

Datenaugmentierung bezeichnet Methoden, durch die die Quantität und Vielfalt der Trainingsdaten künstlich erhöht wird, um die Robustheit und Generalisierungsfähigkeit von Modellen zu verbessern. Insbesondere in Cross-Corpus-Szenarien, in denen verschiedene Korpora innerhalb derselben Domäne hinsichtlich Schreibstil, Textlänge und Annotation variieren, können durch Datenaugmentierung zusätzliche Trainingsbeispiele erzeugt werden, die eine breitere Abdeckung sprachlicher Variationen ermöglichen (J. Wei & Zou, 2019).

Zu den gängigen Techniken der Datenaugmentierung im Bereich der natürlichen Sprachverarbeitung zählen der Synonymaustausch, bei dem Wörter durch ihre semantisch gleichwertigen Alternativen ersetzt werden, sowie zufällige Transformationen wie das Einfügen, Entfernen oder Vertauschen von Wörtern, um unterschiedliche Satzstrukturen zu generieren (Fadaee et al., 2017; J. Wei & Zou, 2019).

Eine Möglichkeit qualitativ hochwertige Paraphrasen zu erstellen, ist die Back-Translation. Hierbei wird der Ausgangstext in eine Fremdsprache übersetzt und anschließend wieder zurück in die Ursprungssprache transformiert. Edunov et al. (2018) zeigen, dass dadurch die Ausdruckvielfalt erhöht wird, ohne dass der semantische Gehalt wesentlich verändert wird. Diese Methode hat sich in Studien von Fadaee et al. (2017) und Sennrich et al. (2016) auch als Verbesserung beim Trainieren von Übersetzungsmodellen selbst gezeigt.

Eine weitere Technik ist die Contextual Augmentation, bei der Sprachmodelle wie BERT genutzt werden, um Wörter basierend auf ihrem Kontext durch alternative, semantisch ähnliche Begriffe zu ersetzen. Kobayashi (2018) beschreibt, dass diese Methode zu einer verbesserten Modellleistung in Klassifikationsaufgaben führen kann, da sie feine Variationen in der Ausdrucksweise erfasst. Darüber hinaus präsentieren J. Wei & Zou (2019) in ihrem Konzept der „Easy Data Augmentation (EDA)“ einfache Transformationen vor, wie den zufälligen Austausch, das Einfügen und Entfernen von Wörtern, die in Textklassifikationsaufgaben zu signifikanten Leistungssteigerungen führen.

Insgesamt führt die Integration diverser Datenaugmentierungsmethoden dazu, dass Modelle mit einer Vielzahl sprachlicher Varianten konfrontiert werden. Dies begünstigt die Entwicklung von robusten und generalisierenden Sprachrepräsentationen, welche die internen Unterschiede zwischen verschiedenen Korpora ausgleichen können (Kobayashi, 2018; J. Wei & Zou, 2019).

3.6 Ensemble-Methoden

Ensemble-Methoden verfolgen das Ziel, die Leistung eines Vorhersagemodells durch die Kombination mehrerer Modelle zu verbessern. Anstatt sich auf ein einzelnes Modell zu verlassen, aggregieren Ensembles die Vorhersagen mehrerer individueller Modelle, um eine robustere und stabilere Gesamtausgabe zu erzeugen (Dietterich, 2000).

Diese Technik kann besonders für Szenarien interessant sein, in denen sich die Trainingsdaten hinsichtlich ihrer sprachlichen Eigenschaften, Annotationen oder Struktur unterscheiden. Konkret würde für jeden Datensatz beziehungsweise Korpus ein eigenes Modell trainiert, das anschließend mit den anderen in einem Ensemble zusammengeführt wird. Auf diese Weise lassen sich die jeweiligen Stärken einzelner Modelle nutzen und Schwächen ausgleichen. Im einfachsten Fall handelt es sich bei einem Ensemble um eine Mehrheitsentscheidung (Voting), bei der die Vorhersagen mehrerer Modelle kombiniert werden. Alternativ können auch gewichtete Mittelwerte der Modelloutputs berechnet oder komplexere Verfahren wie Stacking eingesetzt werden, bei denen ein zusätzliches Metamodell lernt, wie die Vorhersagen der Einzelmodelle bestmöglich zusammengeführt werden (Dietterich, 2000; Habernal et al., 2018; L. Wang et al., 2018).

Im Bereich der natürlichen Sprachverarbeitung demonstrieren Tran et al. (2022), dass Leistungsfähigkeit bei der Termextraktion in einem mehrsprachigen und domänenübergreifenden Kontext durch Ensemble-Methoden verbessert werden kann. Durch die Kombination der Ausgaben verschiedener mono- und multilingualer Modelle in einer Mehrheitsentscheidung konnten sie signifikante Verbesserungen erzielt werden (Tran et al., 2022).

Ein weiteres Beispiel liefert die Studie von Chikhi et al. (2023), in der traditionelle Ensemble- und neuronale Netzwerk-basierte NLP-Algorithmen für die Klassifikation von Finanzdaten verglichen wurden. Ihre Ergebnisse zeigen, dass Ensemble-Modelle eine höhere Klassifikationsgenauigkeit erreichen als einzelne Modelle, insbesondere in hierarchischen Multi-Klassen-Szenarien. Auch bei der CrossNER-Evaluation wurde gezeigt, dass Ensembles besser mit zuvor ungesesehenen Datensätzen umgehen können als einzeln trainierte Modelle (Z. Liu et al., 2020).

Ensemble-Methoden erfordern zwar einen höheren Trainingsaufwand, da für jeden Datensatz beziehungsweise Korpus ein separates Modell trainiert wird, das auf dessen spezifische Eigenschaften abgestimmt ist. Die spätere Kombination kann jedoch modular erfolgen und bewirkt, dass die datensatzspezifischen Eigenschaften erhalten bleiben, während die gemeinsame Ausgabe ein breiteres Verständnis sprachlicher Variationen reflektiert (Habernal et al., 2018).

3.7 Bewertung der Lösungsansätze

In den vorangegangenen Kapiteln wurden verschiedene potenzielle Ansätze zur Optimierung der Leistungsfähigkeit von Modellen in Cross-Corpus-Umgebungen aufgezeigt. Diese Methoden weisen unterschiedliche Stärken und Schwächen auf, die in diesem Kapitel gegeneinander abgewogen werden sollen.

Transfer Learning nutzt vortrainierte Sprachmodelle, die auf umfangreichen, generischen Textkorpora ein breites Sprachverständnis erlernt haben (Devlin et al., 2019; Gururangan et al., 2020). Der Vorteil dieses Ansatzes liegt in der Übernahme generalisierender Repräsentationen. Allerdings wird häufig kritisiert, dass beim klassischen Transfer-Learning zu viele Parameter angepasst werden, sodass das bereits erworbene allgemeine Wissen verloren gehen kann. Dieses Phänomen ist als „catastrophic forgetting“ bekannt (McCloskey & Cohen, 1989). Zudem führt eine vollständige Anpassung der Gewichte oft dazu, dass Modelle zu stark an die spezifischen Merkmale eines einzelnen Korpus überangepasst werden, wodurch ihre Fähigkeit, über Korpusgrenzen hinweg zu generalisieren, eingeschränkt wird (Gururangan et al., 2020; Pan & Yang, 2010).

Die **Domain Adaption**, die den Transfer-Learning Ansatz erweitert, löst dieses Problem, indem sie beim Training gezielte Mechanismen einsetzt, um Unterschiede zwischen den Datensätzen zu minimieren. Die Implementierung dieser Mechanismen erhöht allerdings den Trainingsaufwand und die Modellkomplexität (Ganin et al., 2017; Sun & Saenko, 2016; Tzeng et al., 2017).

Darüber hinaus legen die Studien von Gulrajani & Lopez-Paz (2020) nahe, dass Domain Adaption nur geringe Verbesserungen zeigt, wenn die Unterschiede zwischen den Datensätzen subtil sind. Dies bedeutet das ein Modell nur schwer kleine, aber signifikante Unterschiede erkennen kann.

Multitask Learning betrachtet verschiedene Korpora als verwandte, aber leicht unterschiedliche Aufgaben und trainiert ein gemeinsames Modell simultan auf allen Datensätzen (Caruana, 1997; X. Liu et al., 2019). Dieser Ansatz fördert das Erlernen gemeinsamer Repräsentationen, birgt jedoch die Gefahr, dass korpuspezifische Nuancen verwässert werden. Der Zugewinn an Generalisierungsfähigkeit bleibt daher oft gering, wenn die Unterschiede zwischen den Korpora weiterhin relevant sind (Søgaard & Goldberg, 2016). Zudem können Interferenzen zwischen den Aufgaben auftreten, was zu uneinheitlichen Vorhersagen führen kann (Søgaard & Goldberg, 2016).

Datenaugmentierung vergrößert die Trainingsbasis durch künstlich erzeugte Varianten der Originaldaten (Kobayashi, 2018; J. Wei & Zou, 2019). Dies erhöht die Vielfalt der Trainingsbeispiele, adressiert aber nicht zwangsläufig das Kernproblem, dass Modelle zu stark an die spezifischen Eigenschaften einzelner Korpora angepasst werden. Außerdem können einige Augmentierungsmethoden nur schwer nachvollzogen werden wenn unnatürliche Variationen generiert werden (Fadaee et al., 2017).

Ensemble-Methoden kombinieren die Vorhersagen mehrerer Modelle, um eine robustere Gesamtleistung zu erzielen (Dietterich, 2000; Opitz & Maclin, 1999). In Cross-Corpus-Szenarien können Ensembles die Stärken einzelner, spezifischer Modelle vereinen. Der Ressourcenaufwand ist jedoch signifikant. Darüber hinaus hier kein allgemeines Sprachverständnis aufgebaut, sondern lediglich die Vorhersagen aggregiert. Das bedeutet, dass zukünftige, unbekannte Dokumente, welche sich in ihren Eigenschaften von den Trainingsdaten unterscheiden, nicht notwendigerweise von einem Ensemble profitieren (Chikhi et al., 2023).

Adapter Layers stellen eine Methode dar, die in der jüngsten Forschung als vielversprechende Erweiterung des Transfer Learning diskutiert wird. Im Gegensatz zu Ansätzen, die auf eine umfassende Anpassung des gesamten Modells setzen, integrieren Adapter Layers kleine zusätzliche Module in das vortrainierte Modell (Houlsby et al., 2019). Dadurch werden die Hauptgewichte, die das allgemeine Sprachverständnis repräsentieren, weitgehend unverändert gelassen, was das Risiko des „catastrophic forgetting“ reduziert.

Gleichzeitig ermöglichen sie eine flexible und parameter-effiziente Anpassung an unterschiedliche Datensätze, ohne das generelle Sprachverständnis zu beeinträchtigen. Die einzelnen Adapter können dann flexibel in ein gemeinsames Modell integriert werden (Pfeiffer, Rücklé et al., 2020).

Ein Nachteil der Adapter-Layers ist die Sensibilität bei der Wahl von Hyperparametern. Aufgrund der geringen Anzahl an Parametern können ungünstige Konfigurationen bezüglich Größe und Positionierung schneller wie bei anderen Methoden dazu führen, dass nicht ausreichend datensatzspezifische Merkmale gelernt werden (Houlsby et al., 2019). Dennoch zeigen aktuelle Studien aus Cross-Lingual- und Cross-Domain-Szenarien, dass die Vorteile in Bezug auf Flexibilität, Ressourceneffizienz und Erhaltung vortrainierter Repräsentationen die potenziellen Nachteile überwiegen (Pfeiffer, Vulić et al., 2020).

Ausgehend von dieser Bewertung konzentriert sich die weitere Arbeit auf die Untersuchung von Adapter-Layers im Kontext des beschriebenen Cross-Corpus-Problems.

4 Methodik

In diesem Kapitel wird die experimentelle Methodik erläutert, die in einem kontrollierten Laborexperiment angewendet wird. Außerdem wird der Einsatz des CRISP-DM-Prozesses (Cross Industry Standard Process for Data Mining) als Rahmenkonzept zur systematischen Durchführung der Experimente beschrieben (Wilde & Hess, 2007; Wirth & Hipp, 2000).

4.1 Laborexperiment

Ein Laborexperiment dient dazu, den Einfluss spezifischer, kontrollierbarer Variablen auf das Systemverhalten isoliert zu untersuchen. Im Gegensatz zu Feldexperimenten, bei denen das Modell in einer realen, aber unkontrollierten Umgebung getestet wird, ermöglicht ein kontrolliertes Laborexperiment die Minimierung externer Störfaktoren. Dadurch lassen sich präzisere Aussagen über Ursache-Wirkungs-Beziehungen treffen (Wilde & Hess, 2007).

Für die vorliegende Arbeit werden mehrere Laborexperimente dafür genutzt, um die Effektivität der gewählten Implementierungsmethode „Adapter Layers“ im Umgang mit dem in Kapitel 1.2 formulierten Cross-Corpus-Problem zu evaluieren. Dabei werden zentrale Parameter wie die Adapterkonfiguration, Trainingskonfiguration und Trainingszeit sowie die Positionierung der Adapter im Modell systematisch variiert. Ziel der Experimente ist es, den Einfluss dieser Variablen auf die in Kapitel 2.3 vorgestellten Leistungsmetriken zu messen. Um den direkten Effekt der Adapter-Layers auf die Generalisierungsfähigkeit des Modells isoliert zu untersuchen, werden alle anderen Variablen, insbesondere die Hyperparameter außerhalb der Adapter, konstant gehalten.

Die Experimente werden im Speech & Language Technology Lab des DFKI realisiert, wobei für das Training der GPU-Cluster Pegasus des Instituts verwendet wird.¹ Dabei erfolgt das Training auf zur Laufzeit verfügbaren GPUs, da die Trainingszeit nicht als relevanter Parameter betrachtet wird. Der Fokus liegt auf der Evaluierung der Modellleistung und Generalisierungsfähigkeit.

4.2 CRISP-DM

Der CRISP-DM-Prozess, welcher auch in Abbildung 3 dargestellt ist, bildet einen strukturellen Rahmen für datengestützte Projekte und wird hier als methodischer Leitfaden genutzt, um die Umsetzung der Experimente systematisch zu strukturieren (Wirth & Hipp, 2000). Dieser Prozess gliedert sich in sechs Phasen: Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation und Deployment.

¹ <https://pegasus.dfki.de/docs/slurm-cluster/partitions/>

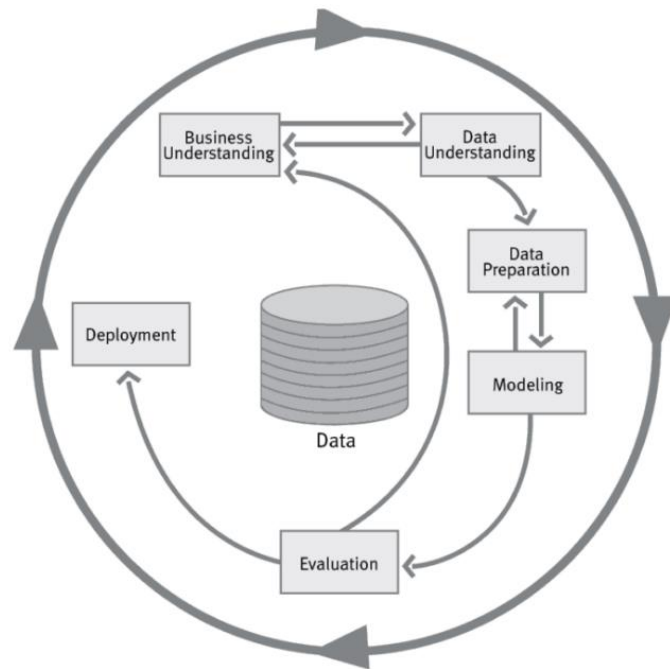


Abbildung 3: CRISP-DM Prozess - Zyklisches Vorgehensmodell von Wirth & Hipp (2000) für Data-Mining-Projekte mit den Phasen Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation und Deployment.

In Kapitel 5.1 erfolgt zunächst die Auswahl von Korpora, die zwar innerhalb derselben Domäne und Sprache vorliegen, jedoch unterschiedliche stilistische und strukturelle Ausprägungen aufweisen sollen. Nachdem diese Korpora identifiziert und fachlich analysiert wurden, beginnt in Kapitel 5.2 die Phase des Data Understanding. Hier werden die stilistischen und strukturellen Eigenschaften der Datensätze untersucht, um die Heterogenität der Daten zu erfassen. Anschließend werden in der Data Preparation Phase im Kapitel 5.3 die Daten durch Standardisierungs- und Normalisierungsverfahren harmonisiert.

Die Modellierungsphase, welche in Kapitel 6 vorzufinden ist, umfasst zunächst die Erstellung einer Baseline, bei der für jeden Korpus ein separates Modell trainiert wird, um diese Modelle anschließend wechselseitig zu validieren. Darauf aufbauend erfolgt die Implementierung der Adapter Layers. In diesem Schritt werden verschiedene Experimente hinsichtlich der Adapterkonfiguration, der Trainingsparameter und der Positionierung der Adapter im Modell durchgeführt. Die Ergebnisse dieser Experimente werden in der Evaluationsphase in Kapitel 7 anhand zentraler Leistungsmetriken ausgewertet, um das Setup mit den besten Ergebnissen zu identifizieren.

Die abschließende Phase des CRISP-DM-Prozesses, das Deployment, wird in dieser Arbeit als optional betrachtet, da der Schwerpunkt auf der experimentellen Analyse und Evaluation liegt.

5 Vorbereitung

In diesem Kapitel erfolgt die Auswahl der Korpora sowie deren Analyse und Vorverarbeitung. Dies bietet die Basis zum Aufbau der Modelle und Experimente im darauffolgenden Kapitel.

5.1 Auswahl Korpora

Wie bereits in Kapitel 4 erwähnt, wird diese Arbeit im Speech & Language Technology Lab des DFKI durchgeführt, wo insbesondere das Cross-Corpus-Problem im Bereich medizinischer Texte beobachtet wurde. Angesichts dieses Forschungskontextes erscheint es naheliegend, Korpora aus dem medizinischen Bereich zu verwenden. Es stellt sich jedoch als eine beträchtliche Herausforderung dar, geeignete NER-Korpora zu identifizieren, die innerhalb derselben Domäne und desselben Anwendungsgebietes angesiedelt sind, aber dennoch unterschiedliche stilistische sowie strukturelle Merkmale aufweisen. Im Rahmen der Recherche konnten zwei Sammlungen von Korpora ermittelt werden, die für die vorliegende Problemstellung in Betracht gezogen werden können.

Mutation Corpora

Diese Sammlung von Thomas & Raithel² umfasst Korpora, die sich auf die Annotation genetischer Mutationen konzentrieren. Die Datensätze zeichnen sich durch konsistente, detaillierte Annotationen aus. Ein Überblick der Sammlung ist Tabelle 1 zuzunehmen.

Korpus	Sprache	Quelle	Entitäten
SETH	Englisch	(Thomas et al., 2016)	Gene, SNP, RS
tmVar	Englisch	(C.-H. Wei et al., 2013)	ProteinMutation, DNAMutation, SNP
AMIA18	Englisch	(Yepes et al., 2018)	Gene_protein, DNA_Mutation, Protein_Mutation, Mutation, DNA_modification, dbSNP, RNA, RNA_Mutation
Variome	Englisch	(Verspoor et al., 2013)	disease, cohort-patient, gene, size, mutation, body-part, Concepts_Ideas, Disorder, Physiology, age, gender, ethnicity, Phenomena
Variome120	Englisch	(Yepes & Verspoor, 2014)	mutation

Tabelle 1: Übersicht über Mutation Corpora - Aufgeschlüsselt nach Sprache, Quelle und enthaltenen Entitätstypen. (nach Thomas & Raithel)

² <https://github.com/Erechtheus/mutationCorpora>

Cross- & Multi-Lingual Medication Detection

Diese Sammlung von Raithel³ umfasst Datensätze, die medikamentenbezogene Informationen in mehreren Sprachen enthalten. Hierbei werden verschiedene Entitätstypen annotiert, die sich je nach Sprache und Datensatz unterscheiden.

Korpus	Sprache	Quelle	Entitäten
BRONCO150	Deutsch	(Kittner et al., 2021)	diagnosis, treatment, medication
GERNERMED	Deutsch	(Frei & Kramer, 2022)	Drug, Strength, Route, Form, Dosage, Frequency, Duration
GGPONC v2.0	Deutsch	(Borchert et al., 2022)	Diagnosis or Pathology, Other Finding, Clinical Drug, Nutrient/Body Subs, External Substance, Therapeutic, Diagnostic
Ex4CDS	Deutsch	(Roller et al., 2022)	Condition, DiagLab, Lab Values, Health-State, Measure, Medication, Age, Process, TimeInfo, Other, risk factor, symptom, conclusion
CMED	Englisch	(Mahajan et al., 2021)	Event, Action, Negation, Temporality, Certainty, Actor
Quaero	Französisch	(Névél et al., 2014)	Anatomy, Chemical and Drugs, Devices, Disorders, Geographic Areas, Living Beings, Objects, Phenomena, Physiology, Procedures
DEFT	Französisch	(Grouin et al., 2019)	Physiology, Surgery, Diseases, Drugs, Temporal data, Lab and exam results
CT-EBM-SP	Spanisch	(Campillos-Llanos et al., 2021)	ANAT, CHEM, DISO, PROC
PharmaCoNER	Spanisch	(Gonzalez-Agirre et al., 2019)	NORMALIZABLES, PROTEINAS, UNCLEAR, NO_NORMALIZABLES

Tabelle 2: Übersicht über Cross- & Multi-Lingual Medication Detection Corpora – Aufgeschlüsselt nach Sprache, Quelle und annotierten Entitätstypen (nach Raithel)⁴

Bei der Analyse der beiden Sammlungen wurde festgestellt, dass die annotierten Entitäten erheblich variieren. Eine weiterführende Recherche in den Hugging Face Dataset Collections, etwa in den BigScience Biomedical Datasets⁵, ergab jedoch keine Datensätze, die hinsichtlich unserer Zielsetzung besser geeignet wären.

³ https://github.com/lraithel/cross_ling_drug_ner

⁴ https://github.com/lraithel/cross_ling_drug_ner

⁵ <https://huggingface.co/bigbio>

Während die zweite Sammlung eine größere Anzahl von Korpora zur Verfügung stellt, entstammen diese Datensätze jedoch unterschiedlichen sprachlichen Umgebungen. Um zusätzliche Einflussvariablen durch sprachliche Unterschiede auszuschließen, wird in dieser Arbeit ausschließlich auf Datensätze in einer einheitlichen Sprache zurückgegriffen. Die Fokussierung auf die deutschsprachigen Korpora der zweiten Sammlung, die in der größten Anzahl vorliegen, zeigt zudem, dass hier weniger Überschneidungen zwischen den annotierten Entitäten bestehen als in den Mutation-Corpora. Aus diesen Gründen wird in der vorliegenden Arbeit die Sammlung der Mutation-Corpora als Grundlage gewählt. Die homogene sprachliche Basis und die konsistente Annotation ermöglichen es, die Auswirkungen unterschiedlicher Schreibstile, Textlängen und Annotationskonventionen gezielt zu analysieren, ohne dass zusätzliche sprachliche Variablen den Analyseprozess beeinflussen.

5.2 Analyse/Exploration Korpora

Im Rahmen der Datenexploration wurde ein Python-basiertes Skript entwickelt, um die ausgewählten Korpora systematisch zu analysieren.⁶ Nach dem Herunterladen der Dateien werden die Rohdaten, die im IOB-Format vorliegen, mithilfe eines eigens entwickelten Parsing-Skripts in eine strukturierte Form überführt. Dabei erfolgt eine schrittweise Aufteilung der Texte in Dokumente, Sätze und schließlich in einzelne Tokens mit den jeweils zugehörigen Labels (Entitätstypen). Eine Übersicht über die Anzahl der Dokumente, Sätze und annotierten Entitäten in den einzelnen Datensätzen ist in Tabelle 3 dargestellt.

Datensatz	Typ	Anzahl Dokumente	Anzahl Sätze	Anzahl Entitäten
SETH	Training	504	4007	2674
SETH	Test	126	951	543
Variome	Training	96	1580	5125
Variome	Test	24	372	971
Variome120	Training	96	1580	87
Variome120	Test	24	372	29
Amia	Training	100	1082	1787
Amia	Test	45	500	752
TmVar	Training	334	4067	982
TmVar	Test	166	1942	469

Tabelle 3: Analyse Mutation-Corpora hinsichtlich Größe und Anzahl der Entitäten ⁷

⁶ https://github.com/denis-trtnr/Cross-Corpus-NER/blob/dev/src/data_preprocessing.py

⁷ <https://github.com/denis-trtnr/Cross-Corpus-NER/blob/dev/src/baseline.ipynb>

Zur weiteren Analyse der Datensätze wird jeweils ein Säulendiagramm erstellt, das die Häufigkeit der einzelnen Entitätsklassen visualisiert. Diese Diagramme dienen der Beurteilung der Klassenbalance und helfen dabei zu erkennen, ob bestimmte Klassen unterrepräsentiert sind.

So zeigt beispielsweise die Visualisierung der Klassenverteilung im SETH-Korpus in Abbildung 4, dass Entitäten des Typs „RS“ in den Trainingsdaten kaum vorkommen und in den Testdaten vollständig fehlen.

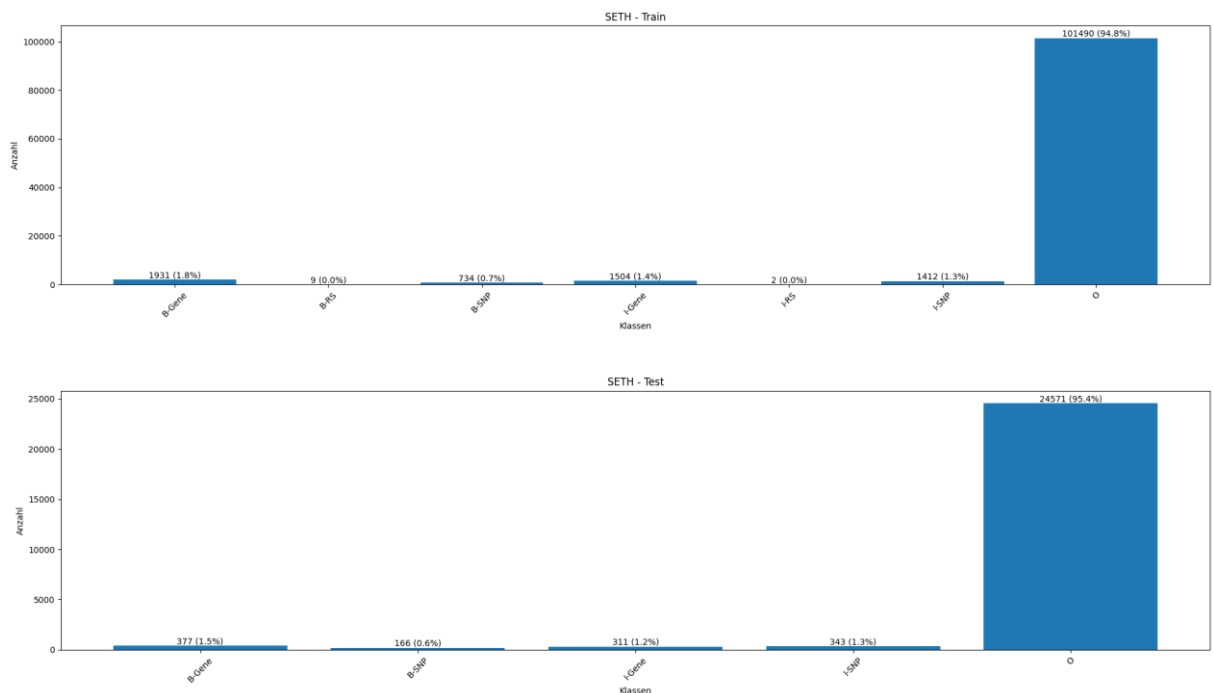


Abbildung 4: Klassenverteilung in SETH-Korpus– Visualisierung der Häufigkeit der Entitätsklassen im Train- (oben) und Test-Set (unten), wobei die Klasse „O“ den Großteil der Tokens ausmacht.⁸

Da die Entität im SETH- als auch in einem anderen Testdatensatz nicht vorkommt und so nie richtig evaluiert werden kann, sollte diese nicht weiter beachtet werden. Dies wird im Schritt der Datenvorverarbeitung vorgenommen. Zudem wurde festgestellt, dass in den Variome-Datensätzen Entitäten wie "Age", "Gender" und "Phenomena" zwar vorhanden sind, jedoch nur selten auftreten. Da diese Entitäten jedoch in Trainings- und Testdatensatz vorkommen, kann eine Zusammenfassung der Attribute sinnvoll sein, um ihre Rolle und Relevanz im Modell zu evaluieren.

Ein weiterer auffälliger Aspekt ist die inkonsistente Benennung gleichartiger Konzepte über die verschiedenen Korpora hinweg. Beispielsweise wird dasselbe Konzept in SETH als "B-Gene" annotiert, in Variome als "B-gene" und in Amia als "B-Gene_protein". Ähnliches gilt für SNPs, die in SETH als "B-SNP" und in Amia als "B-dbSNP" bezeichnet werden. Diese Unterschiede erfordern eine Normalisierung der Labels.

⁸ <https://github.com/denis-trtnr/Cross-Corpus-NER/blob/dev/src/baseline.ipynb>

In der Datenvorverarbeitung soll daher eine Mapping-Struktur implementiert werden, die alle Varianten auf einen einheitlichen Label-Satz abbildet.

Darüber hinaus fällt auf, dass die Granularität der annotierten Entitäten je nach Korpus stark variiert. Während TmVar und Amia detaillierte Unterscheidungen zwischen DNA_Mutation, „RNA_Mutation“ und „ProteinMutation“ treffen, nutzen die Variome-Korpora nur allgemeinere Klassen wie „mutation“. Um diesen Unterschieden zu begegnen, bietet sich ein zweistufiges Mapping an. Zunächst kann eine Normalisierung auf einheitliche, feingranulare Labels erfolgen, bevor diese dann zu abstrakteren Obergruppen zusammengefasst werden. Auch wenn dabei Informationen verloren gehen können, erscheint es sinnvoll zu untersuchen, wie sich die gewählte Granularitätsstufe auf die Modellleistung auswirkt.

5.3 Vorverarbeitung

Die umfassende Analyse und Exploration der Korpora im vorherigen Kapitel liefern nicht nur einen quantitativen Überblick über die Daten, sondern zeigt auch, in welchen Bereichen eine Harmonisierung notwendig ist, um die Heterogenität der Annotationen zu reduzieren und eine konsistente Grundlage für die anschließende Modellierung zu schaffen.

Das angesprochene Mapping bildet dabei den ersten Schritt der Datenvorverarbeitung und ist zusammengefasst in Abbildung 5 dokumentiert.

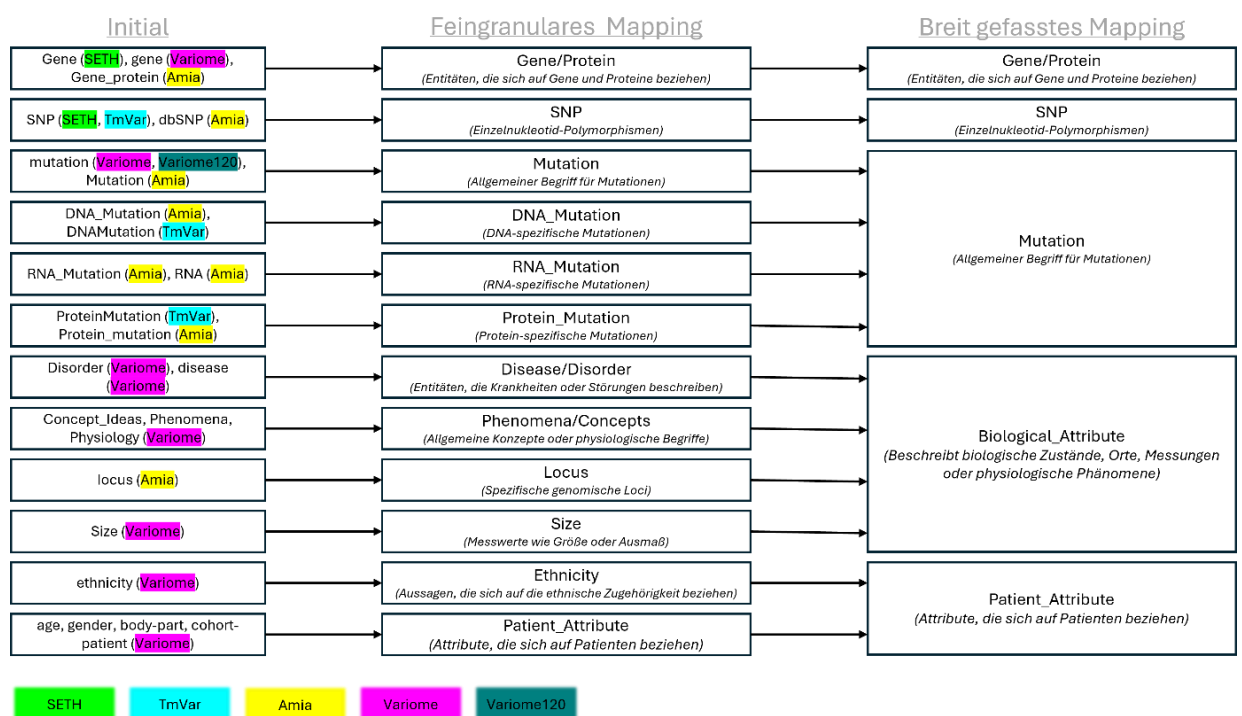


Abbildung 5: Label-Mapping- Übersicht der initialen Entitäten aus den Mutation-Korpora und deren Zuordnung zu fein- und breit gefassten Klassen.⁹

⁹ https://github.com/denis-trtnr/Cross-Corpus-NER/blob/dev/src/data_preprocessing.py

Nach dem Zuordnen der normalisierten Labels beginnt ein mehrstufiger Vorverarbeitungsprozess der Daten. Zunächst werden die Trainingsdaten jedes Korpus zusätzlich in einen Trainings- und Entwicklungsanteil aufgeteilt. Diese sogenannte Train-Dev-Aufteilung erfolgt zufällig im Verhältnis 80 zu 20. Der Entwicklungsdatensatz (Dev-Set) dient dabei der laufenden Evaluierung während des Trainings, ohne die Modellanpassung zu beeinflussen. Dies ist notwendig, um eine unabhängige Einschätzung der Generalisierungsfähigkeit während des Trainingsverlaufs zu erhalten und Overfitting frühzeitig zu erkennen (Goodfellow et al., 2016).

Im nächsten Schritt werden die Trainings-, Dev- und Testdaten mithilfe der Dataset-Klasse aus der Hugging Face Datasets-Bibliothek in ein einheitliches Format überführt (HuggingFace, 2025a). Ein Hugging Face Dataset ist eine abstrahierte Datenstruktur, die eine einheitliche API für den Zugriff, das Mapping, die Transformation und die Batch-Verarbeitung von NLP-Daten bereitstellt. Diese Struktur ermöglicht eine effiziente und skalierbare Datenverarbeitung, insbesondere in Kombination mit dem Trainer-Framework von Hugging Face (HuggingFace, 2025d). Jedes Dataset enthält dabei eine Liste der Tokens eines Satzes und eine Liste der zugehörigen numerischen Entitätslabels (`ner_tags`).¹⁰

Für die Tokenisierung wird ein sogenannter Tokenizer verwendet, der auf demselben Basismodell basiert, das auch im Training eingesetzt wird. Dieser Zusammenhang ist essenziell, da Tokenizer und Modell auf identische Subwort-Vokabulare und Tokenisierungsstrategien abgestimmt sein müssen (HuggingFace, 2025c). Da moderne Transformer-Modelle wie BERT auf Subword-Ebene arbeiten, entstehen durch die Tokenisierung potenziell mehrere Tokens pro ursprünglichem Wort (Devlin et al., 2019).

Dies führt zu einer Schieflage zwischen Token und Label, da das IOB-Label ursprünglich wortweise vergeben wurde. Mithilfe der `align_labels`-Funktion wird dieses Problem adressiert. Hierbei wird jedem Token das entsprechende Label seines Ausgangswortes zugewiesen.¹¹ Bei Subtokens, die nicht am Wortanfang stehen, wird ein B-Label (z. B. B-Gene) gemäß der IOB-Konvention automatisch in ein I-Label (z. B. I-Gene) umgewandelt.

Spezielle Tokens wie `[CLS]` und `[SEP]` erhalten das Label -100, da sie im Trainingsprozess ignoriert werden und keinen Einfluss auf die Berechnung des Verlusts im Modell-Training haben sollen (HuggingFace, 2025b).

Ein weiterer wichtiger Bestandteil der Vorverarbeitung ist der Einsatz eines sogenannten Data Collators. Diese Komponente ist für die Erstellung von Mini-Batches während des Trainings zuständig. Dabei übernimmt sie auch das Padding der Sequenzen auf eine einheitliche Länge und stellt sicher, dass auch die zugehörigen Label-Sequenzen entsprechend angepasst werden (HuggingFace, 2025b).

¹⁰ https://github.com/denis-trtnr/Cross-Corpus-NER/blob/dev/data_preprocessing.py

¹¹ https://github.com/denis-trtnr/Cross-Corpus-NER/blob/dev/data_preprocessing.py

Das Padding der Labels erfolgt ebenfalls mit dem Wert -100, damit auch diese Positionen beim Berechnen der Verlustfunktion ignoriert werden. So wird sichergestellt, dass das Modell nur auf tatsächlich vorhandene Label-Tokens optimiert wird.

6 Modellierung

In diesem Kapitel wird die Modellierungsphase des CRISP-DM-Prozesses im Rahmen der durchgeführten Experimente beschrieben. Der Fokus liegt auf der technischen und methodischen Umsetzung der trainierten NER-Modelle, ohne dabei auf konkrete Ergebnisse oder Leistungsmetriken einzugehen. Eine detaillierte Auswertung und Diskussion der erzielten Resultate erfolgt im anschließenden Kapitel 7.

Zunächst wird in Kapitel 6.1 beschrieben, wie eine Baseline implementiert wird, die als Vergleichsmaßstab für die nachfolgenden Experimente mit Adapter Layers dient. Aufbauend darauf folgt in Kapitel 6.2 die Darstellung der Adapter-Implementierung, bevor in Kapitel 6.3 die in dieser Arbeit durchgeführten Trainings- und Testszenarien erläutert werden.

6.1 Erstellung Baseline

Für jede der ausgewählten Korpora wird ein separates Modell trainiert, das anschließend sowohl auf dem eigenen Testdatensatz als auch auf den Testdaten der jeweils anderen Korpora evaluiert wird. Das Finetuning erfolgt unter Verwendung der Trainer-API aus der Hugging Face Transformers-Bibliothek (Wolf et al., 2020). Dabei werden einheitliche Trainingsparameter über alle Modell-Korpus-Kombinationen hinweg verwendet: Die Lernrate beträgt $2e-5$, das Training erfolgt über fünf Epochen bei einer Batch-Größe von 8. wird das jeweils beste Modell basierend auf der Validierungsperformance gespeichert.¹²

Die Bewertung der Modelleistung orientiert sich an den in Kapitel 2.3 diskutierten Kriterien. Eine vorhergesagte Entität wird nur dann als korrekt gewertet, wenn sowohl die Spanne als auch der Typ exakt mit der Referenzannotation übereinstimmen (Tjong Kim Sang & Meulder, 2003). Insbesondere im medizinischen Kontext ist diese strenge Form der Evaluation essenziell, da bereits kleine Abweichungen potenziell gravierende Fehlschlüsse zur Folge haben können.

Wie in Kapitel 2.3 erläutert, eignen sich die Metriken Precision, Recall und insbesondere der F1-Score in ihrer micro-averaged Form besonders für die Bewertung von NER-Modellen auf stark unausgeglichene Datensätzen. Um diesen Anforderungen gerecht zu werden, kommt in dieser Arbeit das Evaluationsframework segeval zum Einsatz (Hironsan). Diese Bibliothek unterstützt eine Bewertung auf Entitätsebene und berechnet die genannten Metriken standardmäßig auf Basis eines micro-averaged Entity-Level-Ansatzes. Dabei werden alle korrekt, falsch oder fehlklassifizierten Entitäten unabhängig von ihrem Typ gemeinsam aggregiert und gewichtet.

¹² https://github.com/denis-trtnr/Cross-Corpus-NER/blob/dev/src/train_baseline.py

Häufiger vorkommende Klassen fließen entsprechend stärker in die Bewertung ein, wodurch die ungleiche Klassenverteilung adäquat berücksichtigt wird (Tjong Kim Sang & Meulder, 2003).

Da das zentrale Ziel dieser Arbeit die Analyse der Generalisierungsfähigkeit ist, wird jedes trainierte Modell nicht nur auf dem ursprünglichen Testset, sondern zusätzlich auf allen übrigen Korpora evaluiert. Die dabei berechneten entity-level micro-averaged F1-Scores werden anschließend über alle Testdatensätze hinweg gemittelt, um eine zusammenfassende Leistungskennzahl zu erhalten. Diese macro-average über die Testsets ermöglicht eine kompakte Darstellung der Modellperformance im Cross-Corpus-Setting, wobei alle Korpora gleich gewichtet werden. Die resultierende Kennzahl dient als zentrale Referenzgröße, anhand derer spätere Implementierungen, insbesondere jene mit Adapter Layers, vergleichend eingeordnet werden können.

6.2 Adapter Layer Implementierung

Im Gegensatz zu herkömmlichem Finetuning, bei dem das gesamte vortrainierte Sprachmodell wie in Kapitel 6.1 an eine spezifische Aufgabe angepasst wird, erlauben Adapter Layers eine selektivere Anpassung. Der Kerngedanke besteht darin, zusätzliche Netzwerkschichten selektiv in den vorhandenen Encoder-Strukturen unterzubringen. Während die Hauptgewichte des Transformermodells eingefroren bleiben, werden ausschließlich die Parameter dieser Adapter-Layer aktualisiert (Houlsby et al., 2019).

Für die Umsetzung dieser Technik wird in der vorliegenden Arbeit das AdapterHub-Framework eingesetzt (Poth et al., 2023). Dieses Framework bietet eine standardisierte und zugleich hochgradig flexible Möglichkeit, Adapter Layers in bestehende Transformer-Modelle zu integrieren. Ein wesentlicher Grund für den Einsatz dieses Frameworks ist die starke praktische Entlastung, die es bei der Modellimplementierung, -verwaltung und -erweiterung bietet. Zahlreiche technische Aspekte wie die Einbindung der Adapter in die Modellarchitektur, das dynamische Aktivieren und Deaktivieren einzelner Adapter, die Kombination mehrerer Adapter oder die Integration mit der Hugging Face-Transformers-Bibliothek sind bereits vollständig umgesetzt und abstrahiert (Pfeiffer, Rücklé et al., 2020; Poth et al., 2023). Dadurch wird ein erheblicher Teil des technischen Aufwands, der bei einer manuellen Implementierung notwendig wäre, vermieden.

Durch die enge Verzahnung mit der Hugging Face Transformers-Bibliothek (Wolf et al., 2020) bietet das AdapterHub-Framework eine technisch ausgereifte Infrastruktur, die die Integration und Verwaltung von Adapter Layers erheblich vereinfacht. Dabei ist insbesondere die vollständige Kompatibilität mit der modular aufgebauten Transformers-API von zentraler Bedeutung.

Adapter lassen sich direkt in bestehende vortrainierte Modelle wie BERT, BioBERT oder PubMedBERT einfügen, ohne dass Änderungen an der zugrunde liegenden Architektur notwendig wären. Sie können mit wenigen Codezeilen geladen, trainiert, kombiniert oder zur Laufzeit aktiviert und deaktiviert werden (Pfeiffer, Vulić et al., 2020; Poth et al., 2023).

Ein wesentlicher Vorteil besteht darin, dass alle Funktionen der Transformers-Bibliothek auch in Adapter-basierten Experimenten nahtlos genutzt werden können. Dies betrifft nicht nur den Trainingsprozess über die Trainer-API, sondern auch Komponenten wie Tokenizer, DataCollators, integrierte Evaluation, Logging, automatisches Speichern des besten Modells sowie hardwarebeschleunigtes Training (Wolf et al., 2020). Für die in dieser Arbeit umgesetzten Experimente bedeutet es, dass dieselbe Infrastruktur wie bei der Baseline verwendet werden kann. Insbesondere die Datenvorverarbeitung mit Tokenisierung und Label-Alignment bleibt identisch. Dadurch wird methodische Konsistenz sichergestellt und die Gefahr reduziert, dass Unterschiede zwischen Modellvarianten durch inkonsistente Vorverarbeitung oder abweichende Trainingslogik entstehen.

Darüber hinaus lassen sich Adapter unabhängig vom Basismodell versionieren, speichern und wiederverwenden, was ihre Verwaltung im Cross-Corpus-Setting erheblich erleichtert. In Kombination mit der klaren Trennung zwischen Modellkern, Adapter und Datenstruktur entsteht so ein transparentes und skalierbares Experimentier-Setup (Pfeiffer, Rücklé et al., 2020; Pfeiffer, Vulić et al., 2020).

Technisch gesehen ist das AdapterHub-Framework so aufgebaut, dass Adapter typischerweise im Anschluss an die Feedforward-Komponenten eines jeden Encoder-Blocks eingebunden werden. Das zugrunde liegende Prinzip beruht auf einer sogenannten Bottleneck-Struktur, bei der die Dimension der Token-Repräsentation zunächst stark reduziert, dann nichtlinear transformiert und anschließend wieder auf die Originalgröße zurückprojiziert wird (He et al., 2021; Houlby et al., 2019). Auf diese Weise wird das domänenspezifische Wissen komprimiert aufgenommen und in kompakter Form in den Modellfluss eingebracht, ohne den ursprünglichen Signalpfad grundlegend zu verändern. Formal lässt sich der Mechanismus durch folgende Gleichung beschreiben (AdapterHub, 2025b):

$$h \leftarrow W_{up} \times f(W_{down} \times h) + r$$

Hierbei steht h für die kontextualisierte Repräsentation eines Tokens, wie sie aus einer vorangegangenen Schicht des Transformer-Modells kommt. Dieser Vektor wird als Eingabe in den Adapterfluss übergeben. Zunächst erfolgt eine Projektion in eine niedrigere Dimensionalität durch die Down-Projection-Matrix W_{down} , anschließend wird der resultierende Vektor durch eine nichtlineare Aktivierungsfunktion f transformiert, etwa (Rectified Linear Unit) oder GELU (Gaussian Error Linear Unit). Danach wird der Vektor mithilfe der Matrix W_{up} wieder in den

ursprünglichen Raum rückprojiziert (AdapterHub, 2025b). Die Residualverbindung r sorgt dafür, dass der ursprüngliche Eingangsvektor h zur Adapterausgabe addiert wird, was die Stabilität des Trainings erhöht und den Informationsverlust minimiert. Die Größe des Engpasses, also die Dimension des Bottleneck-Zwischenraums, wird im AdapterHub-Framework nicht direkt angegeben, sondern über den Hyperparameter `reduction_factor` definiert. Dieser gibt an, um welchen Faktor die ursprüngliche Dimension reduziert wird. Bei einem Wert von 16 entspricht die Bottleneck-Dimension somit einem Sechzehntel der Modellgröße (AdapterHub, 2025b; Pfeiffer, Rücklé et al., 2020).

Abbildung 6 zeigt die strukturelle Einbindung der Adaptermechanismen innerhalb eines typischen Transformer-Blocks und macht die Flexibilität des AdapterHub-Frameworks im Detail sichtbar. Gezeigt wird ein Encoder-Block, der sowohl einen Multi-Head Attention-Abschnitt (unterer Teil) als auch einen Feedforward-Abschnitt (oberer Teil) umfasst. In beiden Abschnitten ist jeweils ein Adaptermodul in magenta hervorgehoben, das sich die ursprüngliche Architektur integriert, ohne deren Grundstruktur zu verändern (AdapterHub, 2025b). FF Down entspricht dabei der Down-Projection-Matrix W_{down} , während „non_linearity“ die zwischengeschaltete nichtlineare Transformation beschreibt.

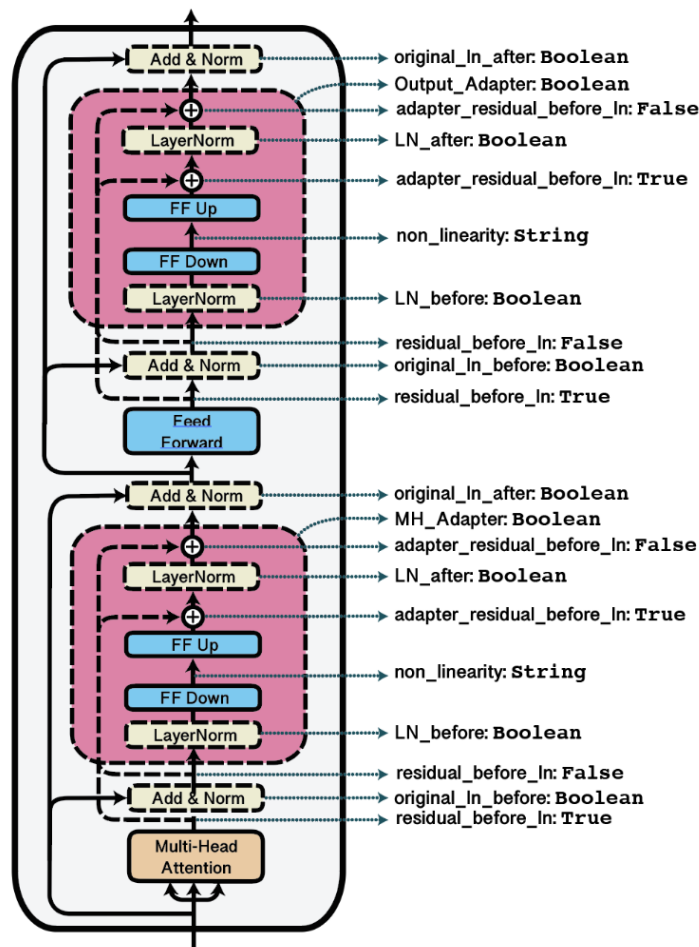


Abbildung 6: Adaptereinbindung und -konfiguration - Darstellung der möglichen Positionen und Parameter für Adapter-Module innerhalb von Transformer-Blöcken. Entnommen aus AdapterHub (2025b)

Die Abbildung macht deutlich, dass das AdapterHub-Framework eine Vielzahl an Konfigurationsmöglichkeiten bietet, mit denen sich das Verhalten und die Platzierung der Adapter präzise steuern lässt. Durch die Kombination dieser Parameter lassen sich unterschiedliche architektonische Varianten definieren, welche in Kapitel 6.3 bei den Trainings- und Testsetups nochmal aufgegriffen werden (Pfeiffer, Vulić et al., 2020). Die Implementierung dieser Konfigurationen ist durch das Framework stark abstrahiert und lässt sich mit wenigen Zeilen Code realisieren.

Neben dem klassischen Bottleneck-Mechanismus existieren weitere Ansätze zur parameter-effizienten Adaption, etwa LoRA (Hu et al., 2021) oder Reft (Wu et al., 2024). Diese Methoden verändern allerdings die Gewichte innerhalb der Attention- oder Feedforward-Matrizen direkt, wodurch sich die resultierenden Modelle nicht mehr modular verwalten, separat speichern oder flexibel kombinieren lassen (AdapterHub, 2025b). Gerade letzteres ist jedoch zentraler Bestandteil des in dieser Arbeit verfolgten experimentellen Designs. Aus diesem Grund wurde bewusst auf alternative Mechanismen verzichtet, auch wenn diese in anderen Kontexten potenziell leistungsfähiger sein mögen.

In der Arbeit wurde eine modulare Trainingspipeline implementiert, die es erlaubt, für jeden Korpus gezielt einen Adapter zu definieren, zu trainieren und anschließend auf alle Testdatensätze zu evaluieren.¹³ Dabei wird für jeden Korpus eine eigene Adapterschicht mit einer beliebig zu definierende Konfiguration zugewiesen. Zusätzlich erhält jeder Adapter einen zugehörigen Prediction Head, also eine separate Ausgabeschicht, die speziell auf die Zielaufgabe NER ausgerichtet ist. Um zu gewährleisten, dass alle Modelle auf denselben sprachlichen Repräsentationen aufbauen und Unterschiede in der Modellleistung ausschließlich auf die Adapter und ihre Generalisierungsfähigkeit zurückzuführen sind, werden die Hauptgewichte des Basismodells eingefroren, bevor das Training des Adapters beginnt.¹⁴

Das Training selbst erfolgt über eine speziell dafür vorgesehene AdapterTrainer-Klasse, die auf den Konzepten des Hugging Face Trainer-Objekts aufbaut, jedoch ausschließlich die Parameter der Adapter und Heads optimiert (AdapterHub, 2025c).

Jeder Adapter wird nach dem Training gegen alle Testdatensätze der anderen Korpora verglichen, um festzustellen, ob hierbei eine ähnliche Modellleistung wie beim Finetuning existiert. Alle trainierten Adapter werden dann abgespeichert damit sie für Experimente bezüglich gemeinsamer Nutzung abrufbereit sind.

Nach Abschluss des Trainingsprozesses wird jeder Adapter inklusive des zugehörigen Heads gespeichert, um eine spätere Wiederverwendung oder Kombination mit anderen Adaptern in den Experimenten zu ermöglichen.

¹³ https://github.com/denis-trtnr/Cross-Corpus-NER/blob/dev/src/train_adapters.py

¹⁴ https://github.com/denis-trtnr/Cross-Corpus-NER/blob/dev/train_adapters.py

Im Anschluss daran erfolgt eine Evaluation auf allen Testdatensätzen. Ziel dieser Evaluation ist es jedoch nicht, bereits die Generalisierungsfähigkeit der Adapter zu messen. Vielmehr dient sie als Ausgangspunkt für die Überprüfung, ob die Reduktion der trainierbaren Parameter im Vergleich zum vollständigen Finetuning zu Leistungseinbußen führt.¹⁵

Wie bereits in Kapitel 3.7 diskutiert, sind Adapter-Ansätze im Vergleich zum klassischen Finetuning zwar deutlich parametereffizienter, jedoch auch sensibler gegenüber der Wahl der Trainingsparameter. Da nur ein Bruchteil der Modellgewichte aktualisiert wird, wirken sich zentraler Hyperparameter wie Lernrate, Dropout oder Batch-Größe stärker auf den Trainingsverlauf aus. Eine ungeeignete Parametrisierung kann hier bereits dazu führen, dass das Modell keine sinnvollen Repräsentationen lernt oder sich nicht stabil optimiert (Houlsby et al., 2019).

Um diesem Risiko zu begegnen und für jeden Adapter eine möglichst leistungsfähige Ausgangskonfiguration zu finden, wurde in dieser Arbeit ein Hyperparameter Tuning mit Hilfe von Weights & Biases durchgeführt (Weights & Biases, 2025). Dabei werden über eine definierte Parameterraum-Spezifikation automatisch verschiedene Kombinationen trainiert und hinsichtlich ihrer Validierungsperformance verglichen. Der Trainingslauf mit dem höchsten F1-Score auf dem Entwicklungssatz wird anschließend ausgewählt und für die Evaluation sowie spätere Experimente verwendet.

Diese systematische Parameterauswahl ist auch deshalb von zentraler Bedeutung, weil sie die Grundlage für alle nachgelagerten Adapterexperimente bildet. Wenn bereits ein einzelner Adapter unter optimalen Trainingsbedingungen nicht in der Lage ist, eine mit dem klassischen Finetuning vergleichbare Leistung zu erzielen, ist es unwahrscheinlich, dass spätere Strategien wie das Kombinieren mehrerer Adapter oder der Transfer zwischen Korpora zu signifikanten Verbesserungen führen. Die Adapterleistung im isolierten Szenario bildet daher eine wichtige Referenz und eine Art Filter, um potenziell ungeeignete Konfigurationen frühzeitig auszuschließen.

6.3 Trainings- und Test-Setups

Um die Generalisierungsfähigkeit von Adapter Layer im Cross-Corpus-Kontext zu untersuchen, werden im Folgenden vier experimentelle Dimensionen betrachtet: (1) die kombinatorische Nutzung mehrerer Adapter über sogenannte Kompositionsstrategien, (2) der Vergleich unterschiedlicher Adapterkonfigurationen, (3) die Auswahl der vortrainierten Sprachmodelle, sowie (4) die Wirkung alternativer Mappingstrategien bei der Normalisierung der Entitätslabels.

¹⁵ https://github.com/denis-trtnr/Cross-Corpus-NER/blob/dev/train_adapters.py

Kompositionsstrategien

Die erste experimentelle Achse umfasst die verschiedenen Möglichkeiten, mehrere bereits trainierte Adapter innerhalb eines Modells zu kombinieren. Im Kontext des Cross-Corpus-Problems eröffnet dies die Möglichkeit, korpuspezifisches Wissen, das in verschiedenen Adaptern enthalten ist, gleichzeitig zu nutzen. Grundlage hierfür ist die durch das Adapters-Framework bereitgestellte Unterstützung für sogenannte Kompositionsblöcke. In dieser Arbeit werden drei verschiedene Kompositionsformen untersucht: Stack, Fuse und Average.

Bei der Stack-Komposition werden mehrere Adapter sequenziell aufeinander angewendet. Die Ausgaben des einen Adapters stellt dabei die Eingabe für den nächsten dar. Abbildung 7 zeigt eine solche Stack-Komposition, bei der exemplarisch zwei Adaptermodule in grün und blau hinter dem regulären Feedforward-Netz eines Encoder-Blocks eingebunden sind.

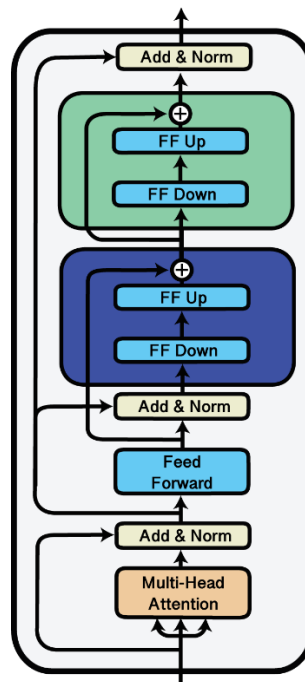


Abbildung 7: Stack-Komposition von Adaptern – Visualisierung zweier seriell geschalteter Adaptermodule in grün und blau („Stacked Adapters“), die nach Feed-Forward-Schicht in den Transformer integriert sind (Poth et al., 2023).

Dieses Prinzip wurde unter anderem im MAD-X-Framework von Pfeiffer, Vulić et al. (2020) erfolgreich für Cross-Lingual-Transfer eingesetzt.

Die zweite Kompositionsstrategie, AdapterFusion (Pfeiffer, Kamath et al., 2020), geht über die sequentielle Verarbeitung hinaus und erlaubt eine parallele Aggregation von Adapterausgaben. Wie in Abbildung 8 visualisiert, wird dabei eine sogenannte Fusionsschicht eingefügt, die auf Self-Attention basiert und eine gewichtete Kombination der Ausgaben mehrerer Adapter ermöglicht.

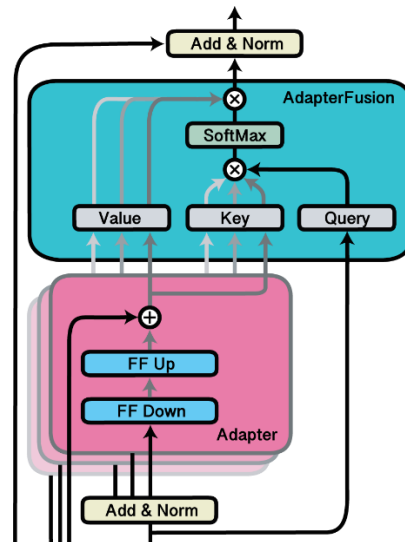


Abbildung 8: Fusion-Komposition von Adaptern - Darstellung der Fusion (blau) mehrerer Adapterausgaben (magenta) über gewichtete Kombinationen im Attention-Mechanismus (Pfeiffer, Kamath et al., 2020).

Dieses Verfahren nutzt den bereits in Kapitel 2.2 erläuterten Self-Attention-Mechanismus, bei dem Token-Repräsentationen anhand Query-, Key- und Value-Vektoren verglichen und über gewichtete Summen miteinander in Beziehung gesetzt werden (Sajun et al., 2024). AdapterFusion überträgt genau dieses Prinzip auf die kontextabhängige Aggregation verschiedener Adapterausgaben. Eine Softmax-Normalisierung stellt sicher, dass die relative Bedeutung der einzelnen Adapterausgaben in einem gewichteten Mittelwert reflektiert wird (Pfeiffer, Kamath et al., 2020). Diese Gewichtung ist lernbar und ermöglicht es dem Modell, im jeweiligen Kontext dynamisch zu entscheiden, welcher Adapter am relevantesten ist. Es handelt sich dabei um ein nicht-destruktives Verfahren, da die ursprünglichen Adapter, welche in Abbildung 8 rosa dargestellt sind, nicht verändert werden. Wichtig ist, dass die Fusionsschicht zusätzlich trainiert werden muss. Nur durch dieses zusätzliche Training kann das Modell lernen, wie die Informationen der einzelnen Adapter im jeweiligen Kontext gewichtet und integriert werden sollen (AdapterHub, 2025a; Pfeiffer, Kamath et al., 2020).

Das dritte Kompositionsverfahren ist Output Averaging, welches Ähnlichkeit zu einer Ensemble Methode hat, wie sie in Kapitel 3.6 beschrieben wurde. Dabei werden die Ausgaben mehrerer Adapter auf Repräsentationsebene lediglich arithmetisch gemittelt. Anders als bei AdapterFusion erfolgt dabei keine dynamische Gewichtung oder Attention-basierte Steuerung, sondern eine feste gleichgewichtete Aggregation (Chronopoulou et al., 2023; X. Wang et al., 2021). Abbildung 9 illustriert die zugrunde liegende Idee, bei der zwei exemplarische Adapter in blau und grün parallel ausgeführt werden und über eine Mittelungsschicht kombiniert werden.

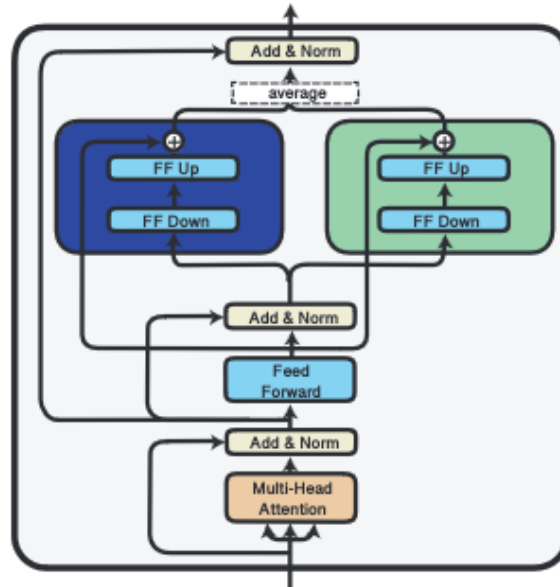


Abbildung 9: Ensemble-Komposition von Adaptern – Zwei Adapterausgaben werden parallel berechnet und anschließend über Mittelung zu einer gemeinsamen Repräsentation zusammengeführt (Chronopoulou et al., 2023). Bild aus Poth et al. (2023).

Da bei Output Averaging keine zusätzlichen Parameter gelernt werden, eignet sich dieses Verfahren insbesondere für den Einsatz mit bereits vollständig trainierten Adaptern. In diesen Experiment wird spannend zu beobachten, inwiefern widersprüchliche Signale zwischen den Adaptern die Modellleistung beeinflussen, da die Methode keine kontextabhängige Gewichtung erlaubt (Chronopoulou et al., 2023).

Alle drei beschriebenen Kompositionsstrategien werden im Rahmen dieser Arbeit experimentell untersucht. Ziel dieser Experimente ist es, zu analysieren, ob und wie sich das Korpora-spezifische Wissen mehrerer Adaptermodule innerhalb eines gemeinsamen Modells kombinieren lässt und wie diese davon profitieren.

Zur Bewertung der Modelle wird ein einheitlicher Evaluierungsansatz verwendet, der auf der in Kapitel 6.1 etablierten Pipeline basiert.¹⁶ Dies stellt sicher, dass Unterschiede in der Modellleistung ausschließlich auf die Kompositionsstrategie und nicht auf veränderte Auswertungsbedingungen zurückzuführen sind. So kann unter identischen Bedingungen überprüft werden, ob die Kombination mehrerer Korpus-spezifischer Adapter zu robusteren Vorhersagen über verschiedene Datenquellen hinweg führt.

¹⁶ https://github.com/denis-trtnr/Cross-Corpus-NER/blob/dev/src/metrics_utils.py

Adapterkonfigurationen

Neben den verschiedenen Möglichkeiten zur Kombination mehrerer Adapter wird in dieser Arbeit auch untersucht, welchen Einfluss unterschiedliche architektonische Ausprägungen der Adaptermodule selbst auf die Modellleistung haben.

Die Auswahl einer geeigneten Adapterkonfiguration ist dabei mit Blick auf die Trainingsstabilität, Parametereffizienz und die spätere Kombinierbarkeit im Cross-Corpus-Szenario relevant. Um diesem Aspekt Rechnung zu tragen, werden im Rahmen der Experimente drei unterschiedliche Konfigurationen gegenübergestellt: Houlsby, Pfeiffer und eine eigens definierte Custom-Konfiguration. Zur besseren Übersicht der Konfigurationen dient Tabelle 4.

	mh_adapter	reduction_factor	non_linearity	dropout	ln_before	ln_after
(Houlsby et al., 2019)	True	16	swish	0.0	True	True
(Pfeiffer, Vulić et al., 2020)	False	16	relu	0.0	True	False
Custom	True	8	gelu	0.1	True	False

Tabelle 4: Genutzte Adapterkonfiguration – Übersicht verschiedener Einstellungen für Multi-Head-Adapter, Reduktionsfaktor, Aktivierungsfunktion, Dropout sowie Layer-Normalisierung, die in den Experimente dieser Arbeit verwendet werden.

Die Houlsby-Konfiguration war eine der ersten systematischen Umsetzungen von Adaptermodulen in Transformer-Architekturen (Houlsby et al., 2019). Sie ergänzt in jedem Transformer-Block ein Adaptermodul nach dem Multi-Head Attention-Modul und eines nach dem Feedforward-Netzwerk. Darüber hinaus wird bei dieser Konfiguration standardmäßig ein sogenannter Multi-Head Adapter verwendet. Dabei handelt es sich nicht um einen Attention-Mechanismus im engeren Sinne, sondern um eine Adaptervariante, bei der mehrere Bottleneck-Pfade parallel berechnet werden. Dies erlaubt es dem Modell, verschiedene Aspekte der Repräsentation simultan zu transformieren und dadurch potenziell vielfältigere Anpassungen vorzunehmen. Ergänzt wird dieses Setup sowohl vor als auch nach dem Adaptermodul durch eine Layer-Normalisierung, welche zu stabileren Trainingsverläufen führen soll.

Die Pfeiffer-Konfiguration (Pfeiffer, Vulić et al., 2020) wurde mit Fokus auf effizienten modularen Wissenstransfer entwickelt. Sie reduziert die Komplexität, indem nur auf ein Adaptermodul nach dem Feedforward-Netz gesetzt wird und verzichtet auf Multi-Head-Adapter. Auch die Layer-Normalisierung wird vereinfacht und nur vor dem Adaptermodul angewendet.

Diese schlankere Struktur erlaubt es, mehrere Adapter effizient zu kombinieren, was sich insbesondere in mehrsprachigen oder domänenübergreifenden Szenarien effektiv gezeigt hat (Pfeiffer, Vulić et al., 2020).

Da das Cross-Corpus-Problem ähnliche Herausforderungen wie domänenübergreifende oder mehrsprachige Szenarien aufweist, orientiert sich die eigens definierte Custom-Konfiguration an der Pfeiffer-Architektur.¹⁷ Sie enthält jedoch gezielte Modifikationen, die auf die spezifischen Anforderungen medizinischer Korpora und das begrenzte Trainingsdatenvolumen abgestimmt sind. So wird Multi-Head Adaption wieder aktiviert, um trotz reduzierter Adapterstruktur eine höhere Ausdrucksstärke zu erzielen. Gleichzeitig wird der `reduction_factor` auf 8 abgesenkt, was eine höhere Repräsentationskapazität im Bottleneck ermöglicht. Die nichtlineare Aktivierungsfunktion wurde auf GELU umgestellt, die sich in kleineren Trainingsszenarien als robuster erwiesen hat (Hendrycks & Gimpel, 2016). Zusätzlich wurde ein moderates Dropout von 0.1 eingeführt, um Überanpassung entgegenzuwirken. Die Konfiguration soll somit eine schlanke Architektur mit gezielter Erweiterung der Modellkapazität kombinieren. Ob diese Anpassungen Vorteile bringen, wird sich in der Evaluation in Kapitel 7 zeigen.

Alle drei vorgestellten Adapterkonfigurationen werden in dieser Arbeit jeweils für das Training korpuspezifischer Adapter verwendet. Die entstandenen Adapter dienen anschließend als Bausteine für die beschriebenen Kompositionsstrategien (Stack, Fuse, Average), wobei innerhalb jeder Komposition ausschließlich Adapter desselben Konfigurationstyps kombiniert werden. Die Experimente sind damit nicht unabhängig voneinander, sondern bauen logisch aufeinander auf.

Jede Adapterkonfiguration wird auch in jeder Komposition getestet, um das Zusammenspiel architektonischer Entscheidungen mit unterschiedlichen Kombinationsansätzen zu untersuchen. Die Evaluation adressiert somit nicht nur die Frage, welche Adapterkonfiguration isoliert am besten funktioniert, sondern auch, wie robust und leistungsfähig sie im Zusammenspiel mit anderen Adaptern in einem domänenübergreifenden Szenario ist.

Auswahl PLM

Für das Training werden zwei vortrainierte Sprachmodelle verwendet: das allgemein trainierte BERT-Base (Devlin et al., 2019) sowie das domänenspezifische PubMedBERT (Aldahdooh et al., 2022). Während BERT-Base auf großen allgemeinen Textsammlungen wie Wikipedia und dem BookCorpus basiert, wurde PubMedBERT speziell für den biomedizinischen Bereich entwickelt.

¹⁷ https://github.com/denis-trtnr/Cross-Corpus-NER/blob/dev/src/train_adapters.py

Das Modell wurde vollständig und ausschließlich auf biomedizinischen Abstracts aus der Datenbank PubMed der U.S. National Library of Medicine trainiert, wodurch es ein besonders tiefes und präzises Sprachverständnis für medizinische Fachsprache entwickelt hat (Aldahdooh et al., 2022; Devlin et al., 2019).

Im Gegensatz zu Modellen, die lediglich durch nachträgliches Finetuning an Fachtexte angepasst wurden, zeichnet sich PubMedBERT durch ein domänenspezifisches Pretraining von Grund auf aus. Laut einer vergleichenden Untersuchung von Madan et al. (2024) zählt es zu den leistungsstärksten Sprachmodellen für medizinisch-biomedizinische Aufgaben. Der Einsatz von PubMedBERT in dieser Arbeit erlaubt eine gezielte Analyse, ob und inwiefern domänenspezifisches Pretraining die Robustheit und Generalisierungsfähigkeit von NER-Modellen im Cross-Corpus-Bereich erhöht. Im Fokus steht dabei auch die Frage, ob ein solches Vortraining die Widerstandsfähigkeit gegenüber stilistischen und strukturellen Unterschieden zwischen medizinischen Korpora verbessert. Beide Sprachmodelle werden in sämtlichen zuvor beschriebenen Experimenten parallel eingesetzt.

Label Mapping

Wie bereits in Kapitel 5.2 aufgezeigt, bestehen zwischen den betrachteten Korpora erhebliche Unterschiede hinsichtlich der Benennung und Granularität der annotierten Entitäten. Um dieser Heterogenität zu begegnen, wurde im Rahmen der Datenvorverarbeitung ein initiales Mapping auf normalisierte Labels durchgeführt, das semantisch äquivalente, jedoch unterschiedlich benannte Entitäten zusammenführt.¹⁸ Zusätzlich wurde eine weiter abstrahierte Mapping-Variante entwickelt, in der detaillierte Entitätstypen auf übergeordnete Kategorien aggregiert werden, um die Auswirkungen unterschiedlicher Granularitätsstufen systematisch untersuchen zu können.

Diese beiden Label-Mappings – eines auf feingranularer, eines auf abstrahierter Ebene – bilden nun die Grundlage für ein experimentelles Vergleichssetup, das analysieren soll, inwiefern die gewählte Labeltiefe die Modellleistung und insbesondere die domänenübergreifende Generalisierungsfähigkeit im Cross-Corpus-Kontext beeinflusst.

¹⁸ https://github.com/denis-trtnr/Cross-Corpus-NER/blob/dev/src/mapping_utils.py

7 Evaluation

In diesem Kapitel werden nun die Ergebnisse der im Kapitel 6.3 beschriebenen Experimente präsentiert. Dafür werden zunächst die Ergebnisse der Baseline evaluiert, bevor die verschiedenen Ergebnisse der Adapterexperimente interpretiert werden.

7.1 Baseline

Die für die weiteren Experimente erstellte Baseline, basierend auf normalem Finetuning einzelner separater Modelle, ist in Tabelle 5 zusammengefasst.

Modell	SETH	Variome	Variome120	Amia	TmVar	Mittelwert
SETH _{BERT}	0,561	0,112	0,000	0,208	0,000	0,176
SETH _{PubMedBERT}	0,690	0,155	0,000	0,355	0,000	0,238
Variome _{BERT}	0,132	0,647	0,034	0,118	0,000	0,184
Variome _{PubMedBERT}	0,161	0,672	0,037	0,162	0,000	0,204
Variome120 _{BERT}	0,000	0,042	0,787	0,000	0,000	0,162
Variome120 _{PubMedBERT}	0,000	0,044	0,829	0,000	0,000	0,171
Amia _{BERT}	0,246	0,143	0,000	0,571	0,032	0,192
Amia _{PubMedBERT}	0,265	0,145	0,006	0,624	0,054	0,215
TmVar _{BERT}	0,000	0,000	0,000	0,032	0,713	0,144
TmVar _{PubMedBERT}	0,000	0,000	0,000	0,038	0,795	0,165
Mittelwert _{BERT}	0,173	0,195	0,160	0,184	0,146	0,171
Mittelwert _{PubMedBERT}	0,211	0,201	0,171	0,251	0,163	0,199

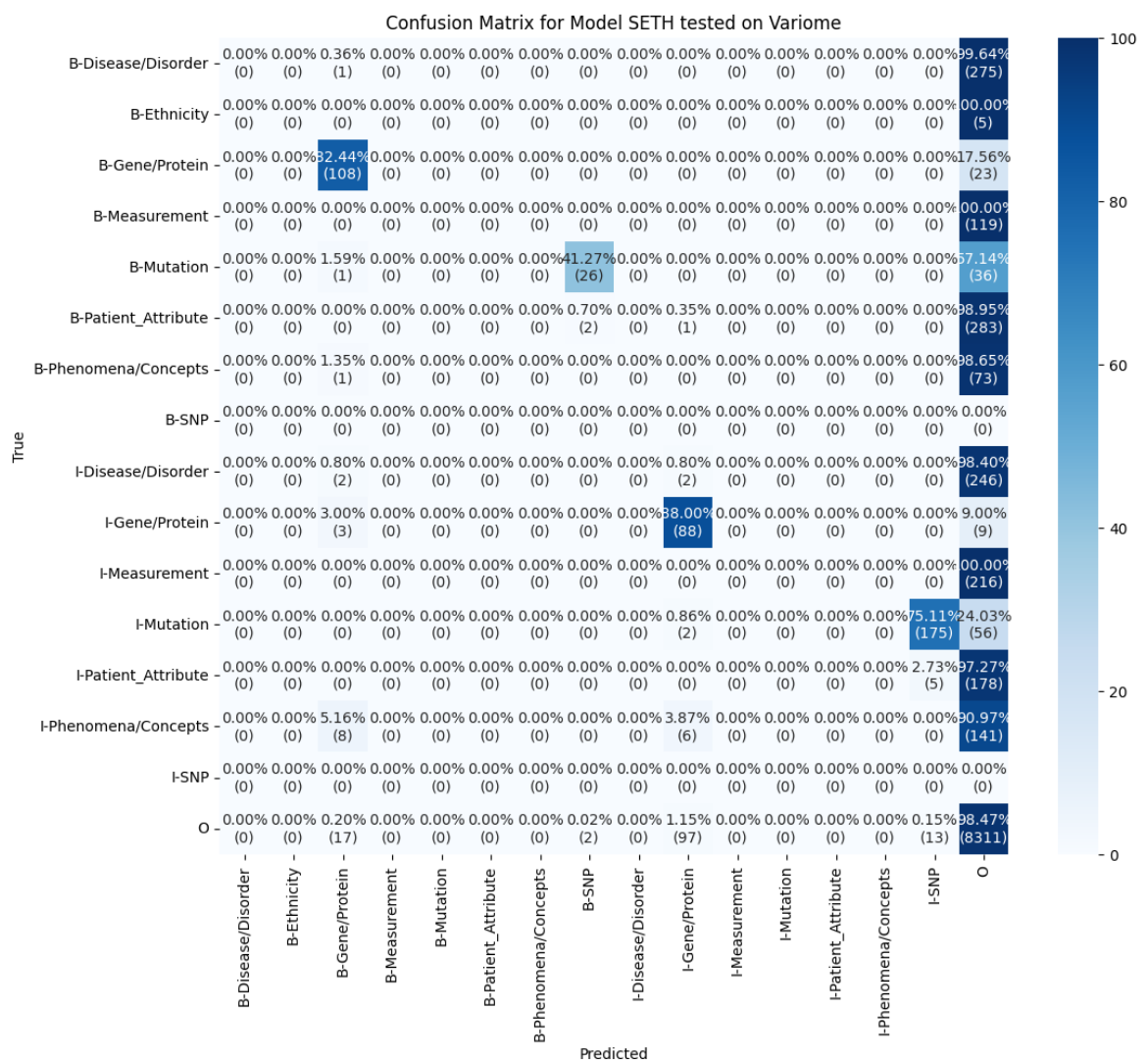
Tabelle 5: Baseline-Ergebnisse: Micro-F1-Scores der pro Korpus trainierten Modelle auf die verschiedenen Testdatensätze der anderen Korpora. Spalten in denen Test- und Trainingsdatensatz aus dem gleichen Korpus stammen, sind grau hervorgehoben. Alle Modelle wurden auf BERT als auch PubMedBERT trainiert. Vor Training wurden die Entitäten mit in Kapitel 5.3 aufgezeigter Funktion fein granular harmonisiert.

Die Ergebnisse zeigen deutliche Unterschiede in der Leistungsfähigkeit zwischen den beiden eingesetzten Modellvarianten. Modelle, die mit PubMedBERT trainiert wurden, erzielen insgesamt konsistent bessere Ergebnisse als jene mit dem Standard-BERT. Dies gilt sowohl für die Evaluation auf dem eigenen Korpus als auch für Tests auf fremden Korpora. Beispielsweise erzielt das Modell SETH _{PubMedBERT} auf dem eigenen Testkorpus einen F1-Score von 0,690, während das entsprechende BERT-Modell bei 0,561 liegt.

Auch auf externen Korpora wie Amia oder Variome zeigen die PubMedBERT-Modelle durchgehend höhere F1-Werte. Dieser Trend zieht sich durch alle getesteten Kombinationen und unterstreicht die Relevanz eines domänenspezifischen Pretrainings für Aufgaben wie NER im biomedizinischen Bereich.

Trotz der insgesamt besseren Leistung der PubMedBERT-basierten Modelle zeigt sich, dass viele Modell-Datensatz-Kombinationen auf externen Korpora nur sehr eingeschränkte Ergebnisse liefern. In einer Vielzahl von Fällen erreichen die Modelle einen F1-Score von 0,000, was bedeutet, dass keine einzige Entität korrekt identifiziert wurde. Diese Beobachtung betrifft vor allem Testszenarien, in denen deutliche Unterschiede zwischen den Trainings- und Testdaten hinsichtlich Annotation, Domänenfokus oder Terminologie bestehen.

Gleichzeitig zeigt sich jedoch, dass unter bestimmten Bedingungen eine begrenzte Generalisierungsfähigkeit möglich ist. Um die Ursachen für diese Beobachtung näher zu untersuchen, wurden Confusion-Matrizen einzelner Tests analysiert, welche detaillierte Einblicke in das Vorhersageverhalten der Modelle auf Entitätsebene ermöglichen. Abbildung 10 zeigt exemplarisch ausgewählte Matrizen.



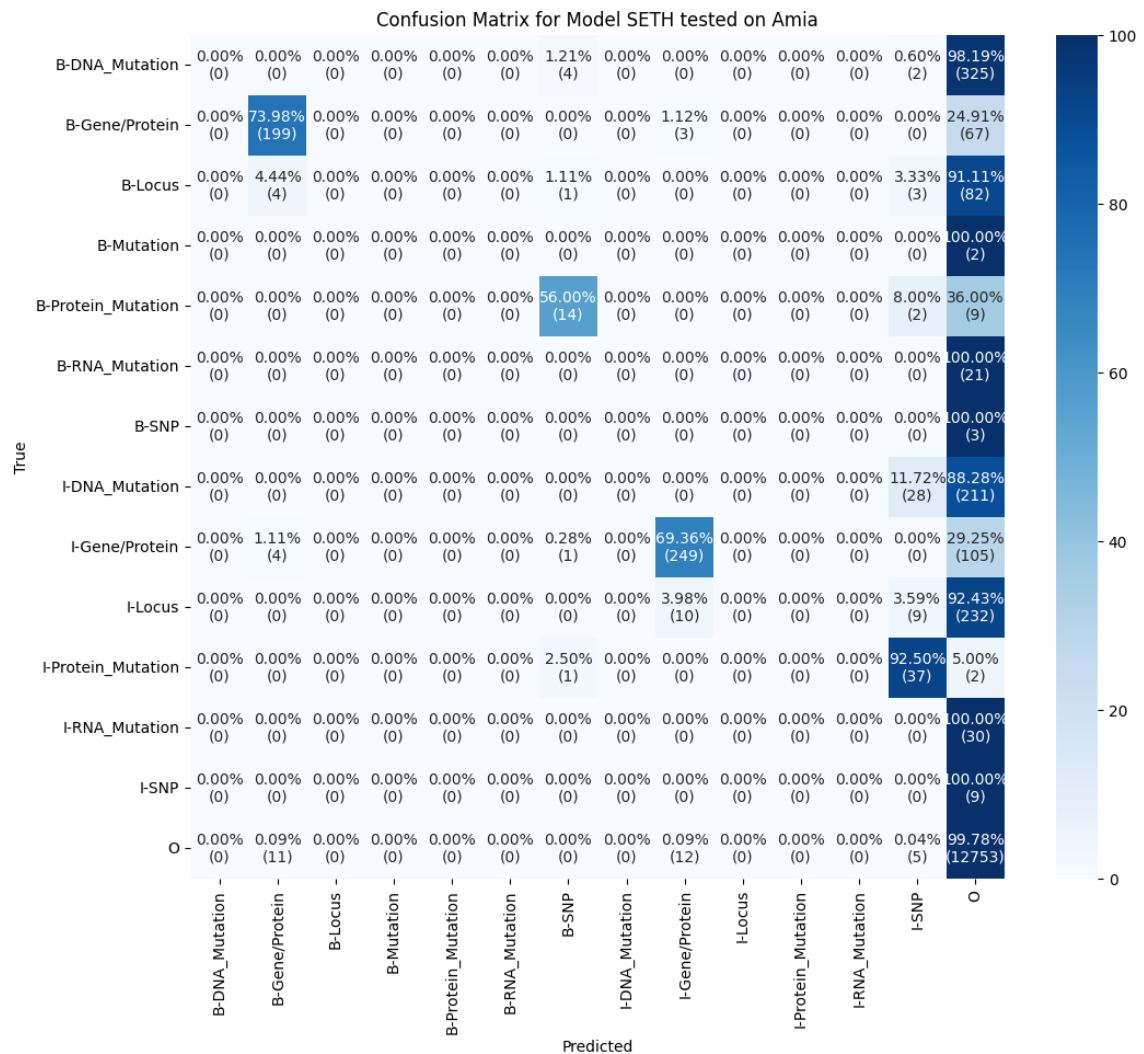


Abbildung 10: SETH-Modell Confusion Matrizen Variome & Amia- Beim Abgleich zwischen vorhergesagten und wirklich richtigen Labels lassen sich korrekte Ergebnisse für Gene/Proteine-Entitäten feststellen.

Die Auswertung der oben dargestellten Confusion-Matrix, in der das auf SETH trainierte Modell auf den Variome-Datensatz evaluiert wird, verdeutlicht, dass beide Korpora die Entität Gene/Protein enthalten. Dies führt zu einer messbaren Übereinstimmung der Vorhersagen auf dieser Entitätskategorie. Ein ähnlicher Effekt lässt sich in der Kombination SETH und Amia in der unteren Matrix zu beobachten. Da beide Datensätze Annotationen für Gene/Protein enthalten, kommt es in beide Richtungen – also sowohl bei SETH → Amia als auch Amia → SETH – zu Vorhersageübereinstimmungen im Bereich von 70–80 Prozent. Es kann jedoch nicht automatisch davon ausgegangen werden, dass die gemeinsame Nutzung gleicher Entitätstypen stets zu einer besseren Übertragbarkeit führt. So enthalten sowohl SETH als auch TmVar Annotationen für SNP, dennoch zeigt die entsprechende Confusion-Matrix in Abbildung 11, dass das auf SETH trainierte Modell die SNP-Entität auf TmVar nicht korrekt vorhersagen konnte. Eine Generalisierung hat in diesem Fall nicht stattgefunden.

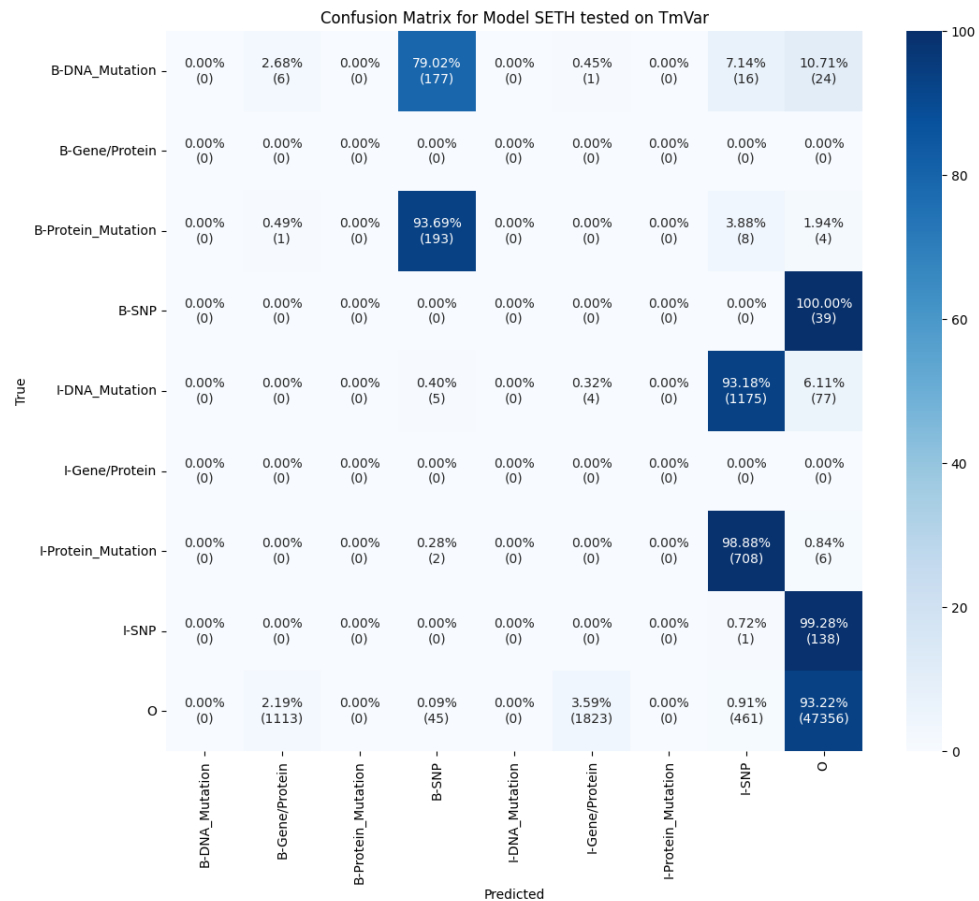


Abbildung 11: SETH-Modell Confusion Matrix TmVar- Beim Abgleich zwischen vorhergesagten und wirklich richtigen Labels lassen sich keine korrekten Vorhersagen für SNP-Entitäten feststellen obwohl in beiden Korpora solche vorkommen.

Insgesamt lässt sich festhalten, dass zwischen einzelnen Korpora eine gewisse inhaltliche Nähe besteht. In ausgewählten Fällen lassen sich dadurch begrenzte Übertragbarkeitseffekte beobachten. Gleichzeitig macht die insgesamt schwache Modellleistung auf domänenfremden Testdaten jedoch die strukturellen Grenzen des bisherigen Ansatzes deutlich. Die durchschnittlichen F1-Scores von 0,171 für BERT und 0,199 für PubMedBERT bilden in diesem Zusammenhang die Baseline für die in Kapitel 6.3 vorgestellten Experimente.

7.2 Adapter-Experimente

Wie zuvor beschrieben, wurde für jeden der fünf Korpora ein eigener Adapter erstellt, der anschließend in verschiedenen Konfigurationen und Kompositionen getestet wurde. Zuvor war jedoch das Ziel, ein vergleichbares Evaluationsszenario zu schaffen, wie es bereits für die vollständig feinjustierten Modelle durchgeführt wurde. Ein Adapter wurde auf dem Trainingsdatensatz eines Korpus (z. B. SETH) trainiert und anschließend auf die Testdaten der jeweils anderen Korpora angewendet. Die Erwartung war, dass die Ergebnisse in diesem Setup weitgehend denen der Fine-Tuning-Modelle entsprechen. Eine Generalisierbarkeit wurde hier noch nicht erwartet, da die Adapter noch nicht zusammen agieren.

Zur Ermittlung optimaler Trainingsparameter wurde mithilfe von Weights & Biases ein Hyperparameter-Sweep durchgeführt. Dabei wurden für jede Adapterkonfiguration jeweils die besten Werte identifiziert, die in Tabelle 6 zusammengefasst sind.

	Trainingsepochen	Batchgröße	Lernrate
Houlsby	12	16	6e-4
Pfeiffer	12	16	3e-4
Custom	10	16	5e-4

Tabelle 6: Optimale Trainingsparameter je Adapterkonfiguration- Für jede in dieser Arbeit berücksichtigte Adapterkonfiguration wurden mittels Hyperparameter-Sweep die optimalen Trainingsparameter identifiziert

Zum Vergleich: Die vorherigen Fine-Tuning-Modelle wurden mit einer festgelegten Konfiguration trainiert, bestehend aus einer Lernrate von 2e-5, einer Batchgröße von 16 sowie einer Trainingsepochenanzahl von 5. Im Gegensatz dazu zeigen die optimalen Konfigurationen für die Adaptermodelle zum Teil deutlich höhere Werte, insbesondere im Hinblick auf die Lernrate und die Anzahl der Epochen. Dies erscheint auf den ersten Blick überraschend, da Adapter im Vergleich zum vollständigen Modell nur einen Bruchteil der Parameter enthalten und somit weniger Gewichtsadjustierungen erforderlich wären. Die Beobachtung legt jedoch nahe, dass die reduzierte Anzahl trainierbarer Parameter durch eine intensivere Optimierung über längere Trainingsverläufe kompensiert werden muss. Die Ergebnisse der lassen sich Tabelle 7 entnehmen

Modell	SETH	Variome	Variome120	Amia	TmVar	Mittelwert
Baseline BERT	0,173	0,195	0,160	0,184	0,146	0,171
Adapter BERT, Houlsby	0,180	0,192	0,126	0,191	0,144	0,168
Adapter BERT, Pfeiffer	0,178	0,187	0,125	0,189	0,139	0,164
Adapter BERT, Custom	0,180	0,190	0,131	0.192	0,140	0,167
Baseline PubMedBERT	0,211	0,201	0,171	0,251	0,163	0,199
Adapter PubMedBERT, Houlsby	0,228	0,199	0,126	0,238	0.165	0,191
Adapter PubMedBERT, Pfeiffer	0,211	0,193	0,114	0,232	0,159	0,182
Adapter PubMedBERT, Custom	0,222	0,194	0,135	0,221	0,157	0,186

Tabelle 7: Ergebnisse Vergleich Adapter/Modelle - Aggregierte Micro-F1-Scores der der verschiedenen Adapter im Vergleich zu den jeweiligen Baseline-Ergebnissen aus Tabelle 6. Für jeden der fünf Korpora wurde ein Adapter trainiert und anschließend auf die Testdaten der jeweils anderen Korpora angewendet. Die Ergebnisse in der Tabelle entsprechen dem Mittelwert der Ergebnisse der Adapter je Adapterkonfigurationen (Houlsby, Pfeiffer, Custom) und PLM (BERT, PubMedBERT).

Die Ergebnisse verdeutlichen zunächst erneut die höhere Eignung von PubMedBERT im Vergleich zu BERT für die vorliegenden biomedizinischen NER-Aufgaben. Unabhängig davon, ob klassisches Fine-Tuning oder Adapter eingesetzt wurden, erzielte PubMedBERT durchgängig höhere F1-Scores.

Darüber hinaus zeigt sich, dass Adaptermodelle in der Lage sind, mit den vollständig feinjustierten Baselines vergleichbare Leistungen zu erzielen. Während die Baseline-Modelle mit BERT und PubMedBERT durchschnittliche F1-Scores von 0,171 bzw. 0,199 erreichen, liegen die besten Adapterkonfigurationen nur geringfügig darunter.

Zwischen den getesteten Adapterarchitekturen zeigen sich dabei klare Leistungsunterschiede. Die Houlsby-Konfiguration erzielt im Schnitt die besten Ergebnisse und übertrifft in einzelnen Fällen sogar die Baselines. Dies deutet darauf hin, dass die erhöhte strukturelle Komplexität zu einer gesteigerten Ausdruckskraft führt. Im Gegensatz zu den anderen Varianten setzt Houlsby zwei Adaptermodule pro Transformer-Block ein und verwendet eine Multihead-Adaption, die parallele Repräsentationspfade berechnet (Houlsby et al., 2019). Diese tiefere Integration ermöglicht mutmaßlich eine differenziertere Anpassung an die korpuspezifischen Strukturen. Die Pfeiffer-Konfiguration, die nur ein Adaptermodul nach dem Feedforward-Block einfügt und auf parallele Pfade verzichtet, bleibt in ihrer Leistung durchweg hinter Houlsby zurück (Pfeiffer, Vulić et al., 2020).

Die eigens entwickelte Custom-Variante basiert auf der Pfeiffer-Architektur, wurde jedoch gezielt erweitert. Unter anderem wurden die Multihead-Adaption reaktiviert, die Reduktionsrate im Bottleneck verringert und eine alternative Aktivierungsfunktion integriert. Damit kombiniert sie eine kompakte Architektur mit ausgewählten Erweiterungen und positioniert sich sowohl strukturell als auch leistungsmäßig zwischen den beiden etablierten Varianten.

Während diese Ergebnisse bereits die Leistungsfähigkeit einzelner Adapter verdeutlichen, stellt sich im nächsten Schritt die Frage, wie sich mehrere Adapter im Zusammenspiel verhalten. Erst durch die systematische Untersuchung solcher Kombinationen mittels Kompositionsstrategien wie Stack, Fuse oder Average, lässt sich das volle Potenzial modularer Adapternetzwerke abschätzen.

Evaluation der Kompositionsexperimente

Damit werden die ersten drei Dimensionen der in Kapitel 6.3 beschriebenen Experimentreihe aufgegriffen. Die Kompositionsstrategien wurden dabei in Abhängigkeit zu den jeweiligen Adapterkonfigurationen und den zugrunde liegenden PLMs untersucht. Die entsprechenden Ergebnisse sind in Tabelle 8 dargestellt.

	Modell	SETH	Variome	Variome120	Amia	TmVar	Mittelwert
Base	Baseline BERT	0,173	0,195	0,160	0,184	0,146	0,171
Houlsby	Fusion BERT, Houlsby	0,419	0,307	0,115	0,235	0,241	0,261
	Stack BERT, Houlsby	0,016	0,028	0,034	0,069	0,102	0,059
	Average BERT, Houlsby	0,131	0,106	0,017	0,074	0,076	0,086
Pfeiffer	Fusion BERT, Pfeiffer	0,432	0,324	0,076	0,231	0,238	0,258
	Stack BERT, Pfeiffer	0,012	0,033	0,010	0,093	0,140	0,058
	Average BERT, Pfeiffer	0,134	0,105	0,015	0,081	0,078	0,079
Custom	Fusion BERT, Custom	0,463	0,321	0,076	0,279	0,242	0,282
	Stack BERT, Custom	0,013	0,037	0,012	0,104	0,160	0,064
	Average BERT, Custom	0,151	0,117	0,019	0,089	0,085	0,093
Base	Baseline PubMedBERT	0,211	0,201	0,171	0,251	0,163	0,199
Houlsby	Fusion PubMedBERT, Houlsby	0,512	0,375	0,140	0,288	0,294	0,319
	Stack PubMedBERT, Houlsby	0,020	0,032	0,044	0,084	0,124	0,073
	Average PubMedBERT, Houlsby	0,160	0,130	0,021	0,090	0,094	0,105
Pfeiffer	Fusion PubMedBERT, Pfeiffer	0,484	0,362	0,087	0,253	0,266	0,289
	Stack PubMedBERT, Pfeiffer	0,014	0,038	0,012	0,105	0,159	0,066
	Average PubMedBERT, Pfeiffer	0,152	0,120	0,018	0,092	0,088	0,090
Custom	Fusion PubMedBERT, Custom	0,561	0,389	0,092	0,338	0,292	0,342
	Stack PubMedBERT, Custom	0,016	0,045	0,014	0,126	0,194	0,079
	Average PubMedBERT, Custom	0,182	0,142	0,023	0,108	0,103	0,112

Tabelle 8: Ergebnis Adapter-Layer-Experimente - Aggregierte Micro-F1-Scores der Adapterkompositionen in Abhängigkeit zu verschiedenen Adapterarchitekturen (Houlsby, Pfeiffer, Custom) und PLMs (BERT, PubMedBERT)

Über alle Testszenarien hinweg zeigt sich, dass die Fusion-Komposition mit deutlichem Abstand die besten Resultate erzielt. Für alle drei Adapterkonfigurationen führt Fusion zu einem signifikanten Anstieg der durchschnittlichen F1-Scores. Die besten Resultate kommen aus der Fusion-Komposition von Adaptern mit Custom-Konfiguration auf Basis des PubMedBERT-Modells. Diese erreicht einen durchschnittlichen Micro-F1-Score von 0,342 und übertrifft damit die entsprechende Baseline (0,199) um 71,8 %.

Gleichzeitig schneiden die Kompositionsstrategien Stack und Average systematisch schlechter als die Baseline ab. Stack bleibt mit Mittelwerten zwischen 0,058 und 0,079 deutlich unter den Erwartungen, während Average leicht darüber liegt, aber ebenfalls keine überzeugenden Ergebnisse liefert.

Auffällig bei der Stack-Kompositionsstrategie ist, dass Korpora wie TmVar (F1 von 0,160 bzw. 0,194), deren Adapter am Ende der Stapelreihenfolge stehen, deutlich bessere Werte erzielen als vordere wie SETH (F1 von 0,016 bzw. 0,020). Das spricht dafür, dass der hinterste Adapter die finale Repräsentation dominiert, während frühere kaum Einfluss haben. Damit ist Stack kaum geeignet, um mehrere Wissensquellen gleichberechtigt zu integrieren.

Auch Average bleibt mit Scores zwischen 0,079 und 0,112 deutlich hinter Fusion zurück. Durch das gleichgewichtete Mitteln aller Adapterausgaben fehlt jegliche Kontextsteuerung. Hilfreiche und widersprüchliche Signale werden undifferenziert vermischt.

Ein klarer Trend zeigt sich in der Gegenüberstellung der beiden PLMs. In sämtlichen Konfigurationen liefern die auf PubMedBERT basierenden Varianten durchgängig bessere Ergebnisse als jene mit BERT-Base. Dieser Vorteil war bereits in der Einzeladapter-Evaluation in Tabelle 7 erkennbar, fällt in der aktuellen Phase der Experimente jedoch noch deutlicher auf. Während die beste BERT-Konfiguration (Custom + Fusion) auf einen F1-Score von 0,282 kommt, erreicht das entsprechende Modell mit PubMedBERT 0,342. Dies entspricht einem relativen Leistungsanstieg um 21%. In der vorherigen Evaluationsphase lag der Leistungszuwachs von BERT zu PubMedBERT bei lediglich 11 bis 13 %. Dies könnte darauf hindeuten, dass die domänenspezifischen Sprachrepräsentationen von PubMedBERT nicht nur die Erkennung medizinischer Entitäten erleichtern, sondern auch eine präzisere Kontextmodellierung im Fusion-Layer ermöglichen. Das Modell scheint dadurch besser in der Lage zu sein, relevante Adapter für einen gegebenen Kontext zu identifizieren und gezielt zu gewichten.

Einfluss der Adapterkonfiguration

Besonders hervorzuheben ist die Leistung der eigens entwickelten Custom-Adapterkonfiguration. Diese Variante erzielt in nahezu allen Szenarien die besten Ergebnisse. Im Vergleich zur Pfeiffer-Architektur profitiert die Custom-Variante von gezielten Erweiterungen wie der Reaktivierung von Multi-Head-Adaption, einem reduzierten Bottleneck-Faktor sowie dem Einsatz der GELU-Aktivierungsfunktion. Diese Anpassungen scheinen insbesondere in Zusammenspiel mit Kompositionsstrategien wie Fusion von Vorteil zu sein, da sie eine ausgewogene Balance zwischen Parametereffizienz und Ausdrucksstärke bieten. Es ist jedoch unklar, welchen der Änderungen in der Adapterkonfiguration primär für die Leistungssteigerung verantwortlich ist. Dies müsste in einem separaten Experiment untersucht werden.

Bemerkenswert ist außerdem, dass die Pfeiffer-Architektur in den aktuellen Experimenten deutlich besser abschneidet als noch in der Einzeladapter-Evaluation. Während sie dort im Durchschnitt merklich hinter Houlsby und Custom zurückblieb, erreicht sie nun in der Fusion-Komposition fast gleichwertige F1-Scores. Bei der Evaluierung des Variome-Korpus erzielt sie mit 0,362 sogar das beste Ergebnis aller getesteten Varianten. Dies deutet darauf hin, dass sich die Vorteile der kompakten Pfeiffer-Struktur insbesondere im Zusammenspiel mit gewichtungsbasierten Kompositionsmechanismen besser entfalten. Die reduzierte Komplexität der Architektur könnte dazu beitragen, dass sich die Fusionsschicht einfacher auf dominante, relevante Adapterausgaben fokussieren kann.

Unterschiede zwischen den Korpora

Neben den Architektur- und Modellunterschieden zeigen sich auch klare Unterschiede zwischen den Korpora hinsichtlich ihrer Transferfähigkeit. Besonders positiv fällt der SETH-Korpus auf, der in allen Fusion-Setups deutliche Leistungszuwächse erzielt. Mit einem F1-Score von 0,561 in der besten Konfiguration verdoppelt sich die Leistung im Vergleich zur Baseline beinahe. Auch in weiteren Kombinationen mit Houlsby- und Pfeiffer-Architektur liefert SETH durchgängig hohe Werte. Eine mögliche Erklärung liegt in der breiten inhaltlichen Überlappung mit mehreren anderen Korpora, wodurch SETH von einer Vielzahl kompatibler Adapterausgaben profitieren kann.

Auch Amia profitiert deutlich von der Fusion-Komposition und zeigt in mehreren Konfigurationen robuste Leistungszuwächse. TmVar und Variome verzeichnen ebenfalls Verbesserungen, wenngleich in moderaterem Umfang.

Auffällig ist hingegen das Ergebnis für Variome120, das im Vergleich zu den übrigen Korpora hinter den Erwartungen zurückbleibt. In den meisten Kompositionsvarianten bleiben die F1-Scores hier unterhalb der Baseline. So erreicht das entsprechende PubMedBERT-Baseline-Modell noch einen F1-Score von 0,171, während die beste Fusion-Konfiguration lediglich 0,135 erzielt. Eine mögliche Erklärung dafür könnte in der inhaltlichen Nähe zu anderen Korpora wie Variome liegen, welcher aus dem gleichen Anwendungsgebiet kommt.

Gerade diese thematische Nähe könnte dazu führen, dass der Fusion-Layer den Variome120-Adapter seltener aktiv einbezieht. In einem lernbasierten Fusionsmechanismus wird adaptiv gewichtet, welcher Adapter in welchem Kontext die informativsten Repräsentationen liefert. Wenn zwei Adapter ähnliche Signale erzeugen, könnte das Modell dazu tendieren, sich im Lauf des Trainings auf einen der beiden Adapter zu konzentrieren (Pfeiffer, Kamath et al., 2020).

Im Falle von Variome und Variome120 könnte das dazu führen, dass der Fusion-Layer bevorzugt den Variome-Adapter nutzt, obwohl der Variome120-Adapter ebenfalls relevante Informationen beiträgt. Dadurch wird der Einfluss des Variome120-Adapters in der finalen Repräsentation abgeschwächt, was sich negativ auf die Modellleistung im entsprechenden Korpus auswirkt.

Diese Beobachtungen verdeutlichen, dass Generalisierungseffekte nicht nur durch Architekturentscheidungen oder PLM-Wahl beeinflusst werden, sondern auch stark davon abhängen, wie gut die Annotationen, Entitätstypen und sprachlichen Merkmale der Korpora zueinander passen.

Evaluation der Label-Mappings

Auch wenn das Label-Mapping nicht im primären Fokus der Arbeit liegt, wurde im Rahmen der Datenvorverarbeitung in Kapitel 5.3 ein zusätzliches, gröberes (broad) Label-Mapping erstellt. Anlass hierfür war die Beobachtung, dass sich die Annotationen der Korpora in ihrer Granularität teils deutlich unterscheiden. Um den Einfluss dieser Unterschiedlichkeit auf die Generalisierungsfähigkeit der Modelle zu untersuchen, wurden sowohl die Baselines als auch alle weiteren Experimente erneut mit diesem breiteren (broad) Label-Mapping durchgeführt. Die Ergebnisse daraus sind Tabelle 9 zu entnehmen. Da sich in den vorangegangenen Experimenten gezeigt hat, dass PubMedBERT durchgängig bessere Resultate erzielt als BERT-Base, wurde für diesen Vergleich ausschließlich PubMedBERT berücksichtigt.

	Modell	SETH	Variome	Variome120	Amia	TmVar	Mittelwert
Base	Baseline Granular	0,211	0,201	0,171	0,251	0,163	0,199
Houlsby	Fusion Granular, Houlsby	0,512	0,375	0,140	0,288	0,294	0,319
	Stack Granular, Houlsby	0,020	0,032	0,044	0,084	0,124	0,073
	Average Granular, Houlsby	0,160	0,130	0,021	0,090	0,094	0,105
Pfeiffer	Fusion Granular, Pfeiffer	0,484	0,362	0,087	0,253	0,266	0,289
	Stack Granular, Pfeiffer	0,014	0,038	0,012	0,105	0,159	0,066
	Average Granular, Pfeiffer	0,152	0,120	0,018	0,092	0,088	0,090
Custom	Fusion Granular, Custom	0,561	0,389	0,092	0,338	0,292	0,342
	Stack Granular, Custom	0,016	0,045	0,014	0,126	0,194	0,079
	Average Granular, Custom	0,182	0,142	0,023	0,108	0,103	0,112

Base	Baseline Broad	0,221	0,269	0,255	0,274	0,264	0,257
Houlsby	Fusion Broad, Houlsby	0,527	0,419	0,155	0,349	0,298	0,340
	Stack Broad, Houlsby	0,026	0,062	0,054	0,104	0,124	0,081
	Average PubMedBERT, Houlsby	0,189	0,022	0,028	0,109	0,111	0,124
Pfeiffer	Fusion Broad, Pfeiffer	0,489	0,366	0,091	0,259	0,269	0,292
	Stack Broad, Pfeiffer	0,039	0,039	0,016	0,103	0,162	0,067
	Average Broad, Pfeiffer	0,182	0,143	0,042	0,100	0,095	0,108
Custom	Fusion Broad, Custom	0,575	0,437	0,102	0,407	0,299	0,364
	Stack Broad, Custom	0,016	0,051	0,015	0,152	0,198	0,084
	Average Broad, Custom	0,214	0,183	0,029	0,149	0,121	0,137

Tabelle 9: Ergebnisse für grobes vs. granuläres Label-Mapping - Aggregierte Micro-F1-Scores der Adapterkompositionen in Abhängigkeit zu verschiedenen Adapterarchitekturen (Houlsby, Pfeiffer, Custom) sowie der jeweils eingesetzten Label-Mapping-Strategie. Alle Modelle wurden auf Basis von PubMedBERT trainiert. Die zugrunde liegenden Mappingvarianten (granular, broad) sind in Abbildung 5 dokumentiert.

Die Ergebnisse zeigen, dass das gröbere Label-Mapping in nahezu allen Konfigurationen zu einer leichten Leistungssteigerung führt. Die generelle Leistungsverteilung zwischen den Adapterkonfigurationen und Kompositionsstrategien bleibt dabei erhalten.

Die Custom Konfiguration erzielt weiterhin durchgehend die höchsten Werte, nun in allen Korpora mit Ausnahme von Variome120. Während bei granularer Annotation die Houlsby-Variante in Kombination mit Fusion für TmVar noch die beste Leistung erreichte, wird sie nun von der Custom-Konfiguration übertroffen.

Besonders profitieren die Korpora Variome und Amia, deren Annotationen in der breiteren Mapping Variante in mehreren Fällen auf gemeinsame Klassen abgebildet wurden. Die Reduktion der Labelvielfalt führt in diesen Fällen nicht nur zu einer niedrigeren Vorhersagekomplexität, sondern erhöht auch die Überlappung der Zielklassen mit anderen Korpora. Dies stärkt die Wirkung der kombinierten Adapter und verbessert die Korpora-übergreifende Übertragbarkeit.

Am deutlichsten fällt die Leistungssteigerung jedoch in den Baseline-Szenarien aus. Eine Ausnahme bildet der SETH-Korpus, bei dem der F1 Score trotz Mapping nahezu unverändert bleibt. Dies lässt sich darauf zurückführen, dass bei SETH keine nennenswerte Zusammenführung von Entitätsklassen vorgenommen wurde.

Der vergleichsweise größere Zugewinn bei den Baseline Modellen im Vergleich zu den Fusion Varianten legt nahe, dass letztere bereits im granularen Setting in der Lage waren, feine Unterschiede zwischen den Entitätstypen durch kontextbasierte Gewichtung der Adapter zu überbrücken. Da der Fusion Layer die jeweiligen Adapterausgaben dynamisch kombiniert, kann er unterschiedliche semantische Ausprägungen gezielt miteinander in Beziehung setzen. In einem solchen Szenario bietet ein zusätzlicher Abstraktionsschritt im Labelraum nur noch begrenzten Mehrwert.

Im Gegensatz dazu operieren die Baseline-Modelle ohne eine solche Repräsentationskombination und profitieren stärker davon, wenn die semantische Komplexität durch eine Vereinheitlichung der Zielklassen reduziert wird.

8 Fazit

Diese Arbeit widmete sich der Frage, wie sich die Leistungsfähigkeit und Generalisierungsfähigkeit von NER-Modellen in einem Cross-Corpus-Szenario verbessern lässt. Untersucht wurde dieses Problem exemplarisch im medizinischen Bereich, konkret anhand von Korpora zur Extraktion genetischer Mutationen. Ausgangspunkte waren die Studien von Augenstein et al. (2017) und Fu et al. (2020), die beobachteten, dass konventionelle Finetuning-Ansätze auf heterogenen Korpora selbst innerhalb der gleichen Domäne häufig erhebliche Leistungseinbußen zeigen und somit in realen, domänenübergreifenden Anwendungskontexten nur begrenzt einsetzbar sind. Diese Problematik konnte durch die Baseline-Experimente in Kapitel 7.1 bestätigt werden. Ein auf dem SETH-Korpus trainiertes PubMedBERT-Modell erreichte auf dem zugehörigen Testdatensatz einen F1-Score von 0,690, während es im Mittel über alle fünf in dieser Arbeit betrachteten Korpora nur einen F1-Score von 0,238 erzielte.

Im theoretischen Teil der Arbeit wurde zunächst eine Bewertung verschiedener methodischer Ansätze zur Verbesserung der Generalisierbarkeit vorgenommen. Berücksichtigt wurden dabei Transfer Learning, Domain Adaptation, Multitask Learning, Datenaugmentation und Ensemble-Verfahren. Die Entscheidung fiel letztlich zugunsten der Adapter Layer, da sie im Vergleich zu den übrigen Ansätzen die vielversprechendste Kombination aus Parametereffizienz, Modularität, Wiederverwendbarkeit und Erhalt des vortrainierten Sprachverständnisses bieten.

Im praktischen Teil wurde ein modulares Experimentiersetup entwickelt, in dem für jeden Korpus ein individueller Adapter trainiert wurde. Diese Adapter wurden anschließend in den Kompositionsstrategien Stack, Average und Fusion miteinander kombiniert und auf ihre Generalisierungsfähigkeit getestet. Die Experimente zeigten, dass insbesondere die Kombination aus Fusion-Komposition, der eigens entwickelten Custom-Adapterkonfiguration und dem domänenspezifischen Sprachmodell PubMedBERT zu den besten Ergebnissen führte. Die leistungsstärkste Variante erzielte dabei einen durchschnittlichen Micro-F1-Score von 0,342. Verglichen mit dem F1 Score von 0,199 der besten Finetuning-Baseline entspricht das einer relativen Steigerung von 71,8%. Und dies, obwohl das zugrunde liegende Sprachmodell vollständig eingefroren blieb und lediglich die Adapterkomponenten trainiert wurden.

Zusätzlich wurde untersucht, welchen Einfluss unterschiedliche Varianten der Label-Zuordnung auf die Modellleistung haben. Dabei zeigte sich, dass ein weniger feingranulares Label-Mapping vor allem den Baseline-Modellen hilft, bessere Ergebnisse zu erzielen. Komplexere Kompositionsstrategien wie Fusion profitierten dagegen weniger stark, da sie offenbar bereits im ursprünglichen Setting in der Lage waren, feine Unterschiede zwischen Entitätstypen sinnvoll zu verarbeiten.

8.1 Kritische Reflexion

Neben den gewonnenen Erkenntnissen weist die Arbeit auch eine Reihe von Einschränkungen auf, die bei der Interpretation der Ergebnisse berücksichtigt werden müssen.

Aufgrund des vorgegebenen begrenzten Umfangs der Arbeit wurde im theoretischen Teil auf ein systematisches Literature Review verzichtet. Die Auswahl möglicher Verfahren zur Verbesserung der Generalisierungsfähigkeit von NER-Modellen basierte auf einer klassischen Literaturrecherche. Dadurch besteht die Möglichkeit, dass relevante Arbeiten oder methodische Alternativen übersehen wurden. Auch bei der Auswahl des verfolgten Ansatzes wurden keine vorher festgelegten, objektiv messbaren Bewertungskriterien verwendet, sondern die Entscheidung auf Grundlage einer qualitativen Abwägung getroffen.

Ein weiterer methodischer Aspekt betrifft die Umsetzung der Adapterarchitektur. In dieser Arbeit wurden ausschließlich Bottleneck-Adapter untersucht, da sich diese zuverlässig mit anderen Adaptoren in verschiedenen Kompositionsstrategien integrieren lassen. Andere Varianten, wie beispielsweise LoRA oder ReFT, blieben unberücksichtigt. Dies schränkt die Aussagekraft der Ergebnisse insofern ein, als nicht ausgeschlossen werden kann, dass alternative Adapterarchitekturen vergleichbare oder sogar bessere Resultate erzielen könnten.

Ein zusätzlich limitierender Faktor betrifft die Auswahl und den Zuschnitt der Evaluation. Getestet wurde ausschließlich auf Datensätzen, für die auch ein eigener Adapter trainiert wurde. Es wäre jedoch aufschlussreich gewesen zu analysieren, wie sich das Modell auf bisher unbekannte Korpora derselben Domäne verhält. Die Ergebnisse eines echten Zero-Shot Szenarios, also ohne zuvor trainierten Adapter, wären für die praktische Anwendung noch aussagekräftiger.

Darüber hinaus ist die Domäne der betrachteten Datensätze auf den Bereich der Mutationserkennung in medizinischen Texten beschränkt. Auch wenn die Resultate hier vielversprechend ausfallen, lässt sich nicht ohne Weiteres ableiten, ob ähnliche Effekte auch in anderen thematischen Kontexten oder Fachdomänen auftreten würden.

Bei den Adapterkonfigurationen wurde eine eigene Custom-Variante entworfen, die in den meisten Experimenten die besten Ergebnisse erzielte. Diese Konfiguration kombiniert mehrere gezielte Anpassungen gegenüber den bestehenden Varianten. Dazu zählen die Aktivierung der Multi-Head-Adaption, eine Reduktion des Bottleneck-Faktors zur Erhöhung der Repräsentationskapazität sowie der Einsatz der GELU-Aktivierungsfunktion. Da alle Änderungen gleichzeitig eingeführt wurden, lässt sich jedoch nicht eindeutig bestimmen, welcher Faktor für den Leistungsgewinn verantwortlich war.

Auch das Label-Mapping stellt eine potenzielle Quelle methodischer Unsicherheit dar. Die Zusammenführung feingranularer Entitätstypen in übergeordnete Klassen erfolgte nach bestem Wissen und Gewissen sowie auf Basis der in den jeweiligen Publikationen dokumentierten Annotationen. Eine domänenspezifische Validierung durch medizinische Fachexperten erfolgte jedoch nicht. Entsprechend kann nicht ausgeschlossen werden, dass einzelne Mappingentscheidungen inkonsistent oder interpretationsabhängig ausfielen.

Nicht zuletzt ist anzumerken, dass die gemessenen Leistungszuwächse zwar statistisch signifikant und methodisch relevant sind, jedoch aus praktischer Perspektive noch nicht ausreichen um ein robustes, produktiv einsetzbares Cross-Corpus-NER-System bereitzustellen. Ein durchschnittlicher F1-Score von 0,342 stellt im Vergleich zur Baseline zwar eine Verbesserung von 71,8 Prozent dar, bleibt aber für viele reale Anwendungsfälle im medizinischen Bereich weiterhin zu niedrig, um ohne Nachbearbeitung zuverlässig nutzbar zu sein.

8.2 Ausblick

Die Ergebnisse dieser Arbeit zeigen, dass Adapter-basierte Strategien ein vielversprechender Ansatz darstellen, um die Generalisierungsfähigkeit von NER-Modellen in Cross-Corpus-Umgebungen zu verbessern. Besonders die Kombination aus der Fusion-Komposition, dem domänenspezifischen PubMedBERT-Modell und einer angepassten Adapterkonfiguration führte zu signifikanten Leistungssteigerungen. Gleichzeitig wird deutlich, dass trotz der erreichten Verbesserungen weiterer Forschungsbedarf besteht, um die Ansätze auch in praktischen Anwendungskontexten effizient nutzbar zu machen.

Ein besonders auffälliges Ergebnis war die starke Leistungsdifferenz zwischen BERT und PubMedBERT in der Fusion-Konfiguration. Während in der Einzeladapter-Evaluation lediglich ein relativer Zuwachs von 11 bis 13 Prozent gemessen wurde, stieg dieser in den Fusion-Experimenten auf rund 21 Prozent. Dies legt nahe, dass domänenspezifisch vortrainierte Sprachmodelle nicht nur bei der Entitätserkennung selbst Vorteile bieten, sondern auch die Kontextmodellierung innerhalb komplexer Aggregationsmechanismen wie dem Fusion-Layer wesentlich verbessern können. Zukünftige Arbeiten könnten gezielt untersuchen, inwiefern spezialisierte Sprachmodelle das Zusammenspiel zwischen Adapterausgaben und deren Gewichtung optimieren.

Ein weiteres wichtiges Ergebnis betrifft die Kompositionsstrategie Average. Diese Strategie, bei der alle Adapterausgaben gleichgewichtet gemittelt werden, erzielte in sämtlichen Konfigurationen deutlich schwächere Ergebnisse als Fusion und blieb häufig sogar hinter der Baseline zurück. Die Resultate legen nahe, dass einfache Mittelwertverfahren ohne Kontextsensitivität für Cross-Corpus-NER ungeeignet sind.

Daraus ergibt sich auch eine begrenzte Relevanz klassischer Ensemble-Methoden, da diese typischerweise ähnliche Aggregationsmechanismen nutzen. Zukünftige Arbeiten sollten daher auf kontextadaptive Verfahren fokussieren, die selektiv auf korpusrelevantes Wissen zugreifen können.

Besonders auffällig war das Ergebnis für den Variome120-Korpus, dessen Leistung in den meisten Kompositionsvarianten unterhalb der Baseline blieb. Eine mögliche Erklärung liegt in der inhaltlichen Nähe zu Variome, wodurch der Fusion-Layer tendenziell stärker dessen Adapter bevorzugt. Wenn zwei Adapter sehr ähnliche Signale erzeugen, kann es vorkommen, dass einer im Trainingsprozess systematisch unterrepräsentiert bleibt. Bevor jedoch konkrete Maßnahmen abgeleitet werden, sollte zunächst überprüft werden, ob dieses Verhalten auch in weiteren Konstellationen reproduzierbar ist und ob tatsächlich ein kausaler Zusammenhang zwischen inhaltlicher Ähnlichkeit und Adapterverdrängung besteht. Falls sich dieser Zusammenhang bestätigt, könnten gezielte Regularisierungsmechanismen entwickelt werden, die eine gleichmäßigere Nutzung semantisch naher Adapter fördern.

Darüber hinaus sollte die Generalisierungsfähigkeit auch in Zero-Shot-Szenarien getestet werden, in denen kein spezifischer Adapter für das Zielkorpus existiert. Ebenso wären alternative Adapterarchitekturen wie LoRA oder ReFT interessant, auch wenn deren Integration in Kompositionsstrategien derzeit technisch eingeschränkt ist.

Insgesamt zeigt die Arbeit, dass modulare Modellarchitekturen wie AdapterFusion in Verbindung mit spezialisierten Sprachmodellen einen vielversprechenden Ansatz für Cross-Corpus-NER darstellen. Zukünftige Forschung sollte daran anknüpfen und diese Ansätze methodisch weiterentwickeln und anwendungsnäher validieren.

Literaturverzeichnis

- AdapterHub. (2025a). *Adapter Activation and Composition — AdapterHub documentation*. https://docs.adapterhub.ml/adapter_composition.html
- AdapterHub. (2025b). *Adapter Methods — AdapterHub documentation*. <https://docs.adapterhub.ml/methods.html>
- AdapterHub. (2025c). *Adapter Training — AdapterHub documentation*. <https://docs.adapterhub.ml/training.html>
- Aldahdooh, J., Vähä-Koskela, M., Tang, J. & Tanoli, Z. (2022). Using BERT to identify drug-target interactions from whole PubMed. *BMC bioinformatics*, 23(1), 245. <https://doi.org/10.1186/s12859-022-04768-x>
- Alshammari, N. & Alanazi, S. (2021). The impact of using different annotation schemes on named entity recognition. *Egyptian Informatics Journal*, 22(3), 295–302. <https://doi.org/10.1016/j.eij.2020.10.004>
- Appelt, D., Hobbs, J. R., Bear, J., Israel, D. J. & Tyson, M. (1993). FASTUS: A Finite-state Processor for Information Extraction from Real-world Text. *International Joint Conference on Artificial Intelligence*. <https://www.semanticscholar.org/paper/FASTUS%3A-A-Finite-state-Processor-for-Information-Appelt-Hobbs/75288ecdeb29f093190c1a0130be2d24619238ed>
- Augenstein, I., Derczynski, L. & Bontcheva, K. (2017). Generalisation in named entity recognition: A quantitative analysis. *Computer Speech & Language*, 44, 61–83. <https://doi.org/10.1016/j.csl.2017.01.012>
- Bapna, A. & Firat, O. (2019). Simple, Scalable Adaptation for Neural Machine Translation. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 1538–1548. <https://doi.org/10.18653/v1/D19-1165>
- Borchert, F., Lohr, C., Modersohn, L., Witt, J., Langer, T., Follmann, M., Gietzelt, M., Arrich, B., Hahn, U. & Schapranow, M.-P. (2022). GGPONC 2.0 - The German Clinical Guideline Corpus for Oncology: Curation Workflow, Annotation Policy, Baseline NER Taggers. *Proceedings of the Thirteenth Language Resources and Evaluation Conference*, 3650–3660. <https://aclanthology.org/2022.lrec-1.389/>
- Campillos-Llanos, L., Valverde-Mateos, A., Capllonch-Carrión, A. & Moreno-Sandoval, A. (2021). A clinical trials corpus annotated with UMLS entities to enhance the access to evidence-based medicine. *BMC Medical Informatics and Decision Making*, 21(1), 69. <https://doi.org/10.1186/s12911-021-01395-z>

- Caruana, R. (1997). Multitask Learning. *Machine Learning*, 28(1), 41–75.
<https://doi.org/10.1023/A:1007379606734>
- Chikhi, A., Mohammadi Ziabari, S. S. & van Essen, J.-W. (2023). A Comparative Study of Traditional, Ensemble and Neural Network-Based Natural Language Processing Algorithms. *Journal of Risk and Financial Management*, 16(7), 327.
<https://doi.org/10.3390/jrfm16070327>
- Chronopoulou, A., Peters, M. E., Fraser, A. & Dodge, J. (2023, 14. Februar). *AdapterSoup: Weight Averaging to Improve Generalization of Pretrained Language Models*.
<http://arxiv.org/pdf/2302.07027>
- Chu, C. & Wang, R. (2018). A Survey of Domain Adaptation for Neural Machine Translation. *Proceedings of the 27th International Conference on Computational Linguistics*, 1304–1319. <https://aclanthology.org/C18-1111/>
- Chu, T., Liu, Y., Deng, J., Li, W. & Duan, L. (2022). Denoised Maximum Classifier Discrepancy for Source-Free Unsupervised Domain Adaptation. *Proceedings of the AAAI Conference on Artificial Intelligence*, 36(1), 472–480.
<https://doi.org/10.1609/aaai.v36i1.19925>
- Devlin, J., Chang, M.-W., Lee, K. & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1*, 4171–4186. <https://aclanthology.org/N19-1423/>
- Dietterich, T. G. (2000). Ensemble Methods in Machine Learning. In (S. 1–15). Springer, Berlin, Heidelberg. https://doi.org/10.1007/3-540-45014-9_1
- Edunov, S., Ott, M., Auli, M. & Grangier, D. (2018). Understanding Back-Translation at Scale. *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, 489–500. <https://doi.org/10.18653/v1/D18-1045>
- Fadaee, M., Bisazza, A. & Monz, C. (2017). Data Augmentation for Low-Resource Neural Machine Translation. *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, 567–573.
<https://doi.org/10.18653/v1/P17-2090>
- Frei, J. & Kramer, F. (2022). GERNERMED: An open German medical NER model. *Software Impacts*, 11, 100212. <https://doi.org/10.1016/j.simpa.2021.100212>
- Fu, J., Liu, P. & Neubig, G. (2020). Interpretable Multi-dataset Evaluation for Named Entity Recognition. *Proceedings of the 2020 Conference on Empirical Methods in Natural*

- Language Processing (EMNLP)*. Vorab-Onlinepublikation.
<https://doi.org/10.18653/v1/2020.emnlp-main.489>
- Ganin, Y., Ustinova, E., Ajakan, H., Germain, P., Larochelle, H., Laviolette, F., Marchand, M. & Lempitsky, V. (2017). Domain-Adversarial Training of Neural Networks. *Domain Adaptation in Computer Vision Applications*, 189–209. https://doi.org/10.1007/978-3-319-58347-1_10
- Gartner. (2023). *Hype Cycle for Generative AI, 2023*. <https://www.gartner.com/document/4726631>
- Gaser, M. (2022). *Subword-level Segmentation for Neural Machine Translation of Code-switched Dialectal Egyptian Arabic -English Text*. https://www.researchgate.net/publication/362477674_Subword-level_Segmentation_for_Neural_Machine_Translation_of_Code-switched_Dialectal_Egyptian_Arabic_-English_Text
<https://doi.org/10.13140/RG.2.2.10302.77123>
- Gonzalez-Agirre, A., Marimon, M., Intxaurreondo, A., Rabal, O., Villegas, M. & Krallinger, M. (2019). PharmaCoNER: Pharmacological Substances, Compounds and proteins Named Entity Recognition track. *Proceedings of the 5th Workshop on BioNLP Open Shared Tasks*, 1–10. <https://doi.org/10.18653/v1/D19-5701>
- Goodfellow, I., Bengio, Y. & Courville, A. (2016). *Deep Learning*. MIT Press, Cambridge, MA.
- Grouin, C., Grabar, N., Claveau, V. & Hamon, T. (2019). Clinical Case Reports for NLP. *Proceedings of the 18th BioNLP Workshop and Shared Task*, 273–282.
<https://doi.org/10.18653/v1/W19-5029>
- Gulrajani, I. & Lopez-Paz, D. (2020, 3. Juli). *In Search of Lost Domain Generalization*.
<http://arxiv.org/pdf/2007.01434>
- Gururangan, S., Marasović, A., Swayamdipta, S., Lo Kyle, Beltagy, I., Downey, D. & Smith, N. A. (2020, 23. April). *Don't Stop Pretraining: Adapt Language Models to Domains and Tasks*. <http://arxiv.org/pdf/2004.10964>
- Habernal, I., Wachsmuth, H., Gurevych, I. & Stein, B. (2018). SemEval-2018 Task 12: The Argument Reasoning Comprehension Task. *Proceedings of the 12th International Workshop on Semantic Evaluation*, 763–772. <https://doi.org/10.18653/v1/S18-1121>
- He, J., Zhou, C., Ma, X., Berg-Kirkpatrick, T. & Neubig, G. (2021, 8. Oktober). *Towards a Unified View of Parameter-Efficient Transfer Learning*. <http://arxiv.org/pdf/2110.04366>
- Hendrycks, D. & Gimpel, K. (2016, 27. Juni). *Gaussian Error Linear Units (GELUs)*.
<http://arxiv.org/pdf/1606.08415>

- Hironsan. *segeval: A python framework for sequence labeling evaluation*.
<https://github.com/chakki-works/segeval>
- Hochreiter, S. & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- Houlsby, N., Giurgiu, A., Jastrzebski, S., Morrone, B., Laroussilhe, Q. de, Gesmundo, A., Attariyan, M. & Gelly, S. (2019, 2. Februar). *Parameter-Efficient Transfer Learning for NLP*. <http://arxiv.org/pdf/1902.00751>
- Howard, J. & Ruder, S. (2018). Universal Language Model Fine-tuning for Text Classification. *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 328–339. <https://doi.org/10.18653/v1/P18-1031>
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L [Lu] & Chen, W. (2021, 17. Juni). *LoRA: Low-Rank Adaptation of Large Language Models*. <http://arxiv.org/pdf/2106.09685>
- Huang, Z., Xu, W. & Yu, K. (2015). Bidirectional LSTM-CRF Models for Sequence Tagging. *arXiv.org*. <https://www.semanticscholar.org/paper/Bidirectional-LSTM-CRF-Models-for-Sequence-Tagging-Huang-Xu/af88ce6116c2cd2927a4198745e99e5465173783>
- HuggingFace. (2025a). *Datasets - Bibliothek*. <https://huggingface.co/docs/hub/en/datasets>
- HuggingFace. (2025b). *Token classification*. https://huggingface.co/docs/transformers/tasks/token_classification
- HuggingFace. (2025c). *Token classification - Hugging Face LLM Course*. <https://huggingface.co/learn/llm-course/chapter7/2?fw=pt>
- HuggingFace. (2025d). *Trainer*. https://huggingface.co/docs/transformers/main_classes/trainer
- Jiang, R., Banchs, R. E. & Li, H. (2016). Evaluating and Combining Name Entity Recognition Systems. *Proceedings of the Sixth Named Entity Workshop*, 21–27. <https://doi.org/10.18653/v1/W16-2703>
- Jurafsky, D [Daniel] & Martin, J. (2013). *Speech and Language Processing*. Stanford University. <https://web.stanford.edu/~jurafsky/slp3/>
- Khashabi, D., Min, S., Khot, T., Sabharwal, A., Tafjord, O., Clark, P. & Hajishirzi, H. (2020, 2. Mai). *UnifiedQA: Crossing Format Boundaries With a Single QA System*. <http://arxiv.org/pdf/2005.00700>

- Kittner, M., Lamping, M., Rieke, D. T., Götze, J., Bajwa, B., Jelas, I., Rüter, G., Hautow, H., Sängner, M., Habibi, M., Zettwitz, M., Bortoli, T. de, Ostermann, L., Ševa, J., Starlinger, J., Kohlbacher, O., Malek, N. P., Keilholz, U. & Leser, U. (2021). Annotation and initial evaluation of a large annotated German oncological corpus. *JAMIA Open*, 4(2), ooab025. <https://doi.org/10.1093/jamiaopen/ooab025>
- Kobayashi, S. (2018). Contextual Augmentation: Data Augmentation by Words with Paradigmatic Relations. *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, 452–457. <https://doi.org/10.18653/v1/N18-2072>
- Lafferty, J., McCallum, A. & Pereira, F. (2001). Conditional Random Fields: Probabilistic Models for Segmenting and Labeling Sequence Data. *International Conference on Machine Learning*. <https://www.semanticscholar.org/paper/Conditional-Random-Fields%3A-Probabilistic-Models-for-Lafferty-McCallum/f4ba954b0412773d047dc41231c733de0c1f4926>
- Lample, G., Ballesteros, M., Subramanian, S., Kawakami, K. & Dyer, C. (2016). Neural Architectures for Named Entity Recognition. *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 260–270. <https://doi.org/10.18653/v1/N16-1030>
- Lan, Z., Chen, M., Goodman, S., Gimpel, K., Sharma, P. & Soricut, R. (2019, 26. September). *ALBERT: A Lite BERT for Self-supervised Learning of Language Representations*. <http://arxiv.org/pdf/1909.11942>
- Lecun, Y., Bottou, L., Bengio, Y. & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324. <https://doi.org/10.1109/5.726791>
- Lin, B. Y. & Lu, W. (2018, 15. Oktober). *Neural Adaptation Layers for Cross-domain Named Entity Recognition*. <http://arxiv.org/pdf/1810.06368>
- Liu, X., He, P., Chen, W. & Gao, J. (2019). Multi-Task Deep Neural Networks for Natural Language Understanding. *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 4487–4496. <https://doi.org/10.18653/v1/P19-1441>
- Liu, Z., Xu, Y., Yu, T., Dai, W., Ji, Z., Cahyawijaya, S., Madotto, A. & Fung, P. (2020, 8. Dezember). *CrossNER: Evaluating Cross-Domain Named Entity Recognition*. <http://arxiv.org/pdf/2012.04373>

- Llorca, I., Borchert, F. & Schapranow, M.-P. (2023). A Meta-dataset of German Medical Corpora: Harmonization of Annotations and Cross-corpus NER Evaluation. *Proceedings of the 5th Clinical Natural Language Processing Workshop*, 171–181. <https://doi.org/10.18653/v1/2023.clinicalnlp-1.23>
- Madan, S., Lentzen, M., Brandt, J., Rueckert, D., Hofmann-Apitius, M. & Fröhlich, H. (2024). Transformer models in biomedicine. *BMC Medical Informatics and Decision Making*, 24(1), 214. <https://doi.org/10.1186/s12911-024-02600-5>
- Mahajan, D., Liang, J. J. & Tsou, C.-H. (2021). Toward Understanding Clinical Context of Medication Change Events in Clinical Narratives. *AMIA ... Annual Symposium proceedings. AMIA Symposium, 2021*, 833–842. <https://pubmed.ncbi.nlm.nih.gov/35308981/>
- McCloskey, M. & Cohen, N. J. (1989). Catastrophic Interference in Connectionist Networks: The Sequential Learning Problem. In G. H. Bower (Hrsg.), *Psychology of Learning and Motivation* (Bd. 24, S. 109–165). Academic Press. [https://doi.org/10.1016/S0079-7421\(08\)60536-8](https://doi.org/10.1016/S0079-7421(08)60536-8)
- Nadeau, D. & Sekine, S. (2009). A survey of named entity recognition and classification. *Named Entities*, 3–28. <https://www.degruyter.com/document/doi/10.1075/bct.19.03nad/pdf>
- Névéol, A., Grouin, C., Leixa, J., Rosset, S. & Zweigenbaum, P. (2014). The Quaero French Medical Corpus : A Ressource for Medical Entity Recognition and Normalization. <https://www.semanticscholar.org/paper/The-Quaero-French-Medical-Corpus-%3A-A-Ressource-for-N%C3%A9v%C3%A9ol-Grouin/2383cadffc89e22a9e67162cc22c8993d675e2e9>
- Opitz, D. & Maclin, R. (1999). Popular Ensemble Methods: An Empirical Study. *Journal of Artificial Intelligence Research*, 11, 169–198. <https://doi.org/10.1613/jair.614>
- Pan, S. J. & Yang, Q. (2010). A Survey on Transfer Learning. *IEEE Transactions on Knowledge and Data Engineering*. <https://www.semanticscholar.org/paper/A-Survey-on-Transfer-Learning-Pan-Yang/a25fbcbbae1e8f79c4360d26aa11a3abf1a11972>
- Pfeiffer, J., Kamath, A., Rücklé, A., Cho, K. & Gurevych, I. (2020). AdapterFusion: Non-Destructive Task Composition for Transfer Learning. *Proceedings of EACL*. <http://arxiv.org/pdf/2005.00247>
- Pfeiffer, J., Rücklé, A., Poth, C., Kamath, A., Vulić, I., Ruder, S., Cho, K. & Gurevych, I. (2020). AdapterHub: A Framework for Adapting Transformers. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 46–54. <https://doi.org/10.18653/v1/2020.emnlp-demos.7>

- Pfeiffer, J., Vulić, I., Gurevych, I. & Ruder, S. (2020). MAD-X: An Adapter-Based Framework for Multi-Task Cross-Lingual Transfer. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 7654–7673.
<https://doi.org/10.18653/v1/2020.emnlp-main.617>
- Poth, C., Sterz, H., Paul, I., Purkayastha, S., Engländer, L., Imhof, T., Vulić, I., Ruder, S., Gurevych, I. & Pfeiffer, J. (2023, 18. November). *Adapters: A Unified Library for Parameter-Efficient and Modular Transfer Learning*. <http://arxiv.org/pdf/2311.11077>
- Raithel, L. *Cross- & Multi-Lingual Medication Detection: A Transformer-based Analysis*.
https://github.com/lraithel/cross_ling_drug_ner
- Ramshaw, L. A. & Marcus, M. P. (1999). Text Chunking Using Transformation-Based Learning. In *Natural Language Processing Using Very Large Corpora* (S. 157–176). Springer, Dordrecht. https://doi.org/10.1007/978-94-017-2390-9_10
- Rogers, A., Kovaleva, O. & Rumshisky, A. (2020, 27. Februar). *A Primer in BERTology: What we know about how BERT works*. <http://arxiv.org/pdf/2002.12327>
- Roller, R., Burchardt, A., Feldhus, N., Seiffe, L., Budde, K., Ronicke, S. & Osmanodja, B. (2022). An Annotated Corpus of Textual Explanations for Clinical Decision Support. *Proceedings of the Thirteenth Language Resources and Evaluation Conference*, 2317–2326. <https://aclanthology.org/2022.lrec-1.248/>
- Ruder, S. (2017, 15. Juni). *An Overview of Multi-Task Learning in Deep Neural Networks*. <http://arxiv.org/pdf/1706.05098>
- Saito, K., Watanabe, K., Ushiku, Y. & Harada, T. (2017, 7. Dezember). *Maximum Classifier Discrepancy for Unsupervised Domain Adaptation*. <http://arxiv.org/pdf/1712.02560>
- Sajun, A. R., Zualkernan, I. & Sankalpa, D. (2024). A Historical Survey of Advances in Transformer Architectures. *Applied Sciences*, 14(10), 4316.
<https://doi.org/10.3390/app14104316>
- Sänger, M., Garda, S., Wang, X. D., Weber-Genzel, L., Droop, P., Fuchs, B., Akbik, A. & Leser, U. (2024). HunFlair2 in a cross-corpus evaluation of biomedical named entity recognition and normalization tools. *Bioinformatics (Oxford, England)*, 40(10).
<https://doi.org/10.1093/bioinformatics/btae564>
- Sanh, V., Debut, L., Chaumond, J. & Wolf, T. (2019, 2. Oktober). *DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter*. <http://arxiv.org/pdf/1910.01108>

- Sennrich, R., Haddow, B. & Birch, A. (2016). Improving Neural Machine Translation Models with Monolingual Data. *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 86–96.
<https://doi.org/10.18653/v1/P16-1009>
- Søgaard, A. & Goldberg, Y. (2016). Deep multi-task learning with low level tasks supervised at lower layers. *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, 231–235.
<https://doi.org/10.18653/v1/P16-2038>
- Sun, B. & Saenko, K. (2016, 6. Juli). *Deep CORAL: Correlation Alignment for Deep Domain Adaptation*. <http://arxiv.org/pdf/1607.01719>
- Takahashi, K., Yamamoto, K., Kuchiba, A. & Koyama, T. (2022). Confidence interval for micro-averaged F1 and macro-averaged F1 scores. *Applied Intelligence*, 52(5), 4961–4972. <https://doi.org/10.1007/s10489-021-02635-5>
- Thomas, P. & Raithel, L. *mutationCorpora*. <https://github.com/Erechtheus/mutationCorpora>
- Thomas, P., Rocktäschel, T., Hakenberg, J., Lichtblau, Y. & Leser, U. (2016). SETH detects and normalizes genetic variants in text. *Bioinformatics (Oxford, England)*, 32(18), 2883–2885. <https://doi.org/10.1093/bioinformatics/btw234>
- Tjong Kim Sang, E. & Meulder, F. de (2003). Introduction to the CoNLL-2003 Shared Task: Language-Independent Named Entity Recognition. *Proceedings of the Seventh Conference on Natural Language Learning at HLT-NAACL 2003*, 142–147. <https://aclanthology.org/W03-0419/>
- Tran, H. T. H., Martinc, M., Pelicon, A., Doucet, A. & Pollak, S. (2022). Ensembling Transformers for Cross-domain Automatic Term Extraction. *International Conference on Asian Digital Libraries (ICADL)*, 13636, 90–100. https://doi.org/10.1007/978-3-031-21756-2_7
- Tsai, R. T.-H., Wu, S.-H., Chou, W.-C., Lin, Y.-C., He, D., Hsiang, J., Sung, T.-Y. & Hsu, W.-L. (2006). Various criteria in the evaluation of biomedical named entity recognition. *BMC Bioinformatics*, 7(1), 92. <https://doi.org/10.1186/1471-2105-7-92>
- Tzeng, E., Hoffman, J., Saenko, K. & Darrell, T. (2017, 17. Februar). *Adversarial Discriminative Domain Adaptation*. <http://arxiv.org/pdf/1702.05464>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L. & Polosukhin, I. (2017). Attention Is All You Need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*.

- Verspoor, K., Jimeno Yepes, A., Cavedon, L., McIntosh, T., Herten-Crabb, A., Thomas, Z. & Plazzer, J.-P. (2013). Annotating the biomedical literature for the human variome. *Database : the journal of biological databases and curation*, 2013, bat019. <https://doi.org/10.1093/database/bat019>
- Wacholder, N., Ravin, Y. & Choi, M. (1997). Disambiguation of proper names in text. In *Proceedings of the fifth conference on Applied natural language processing* - (S. 202–208). Association for Computational Linguistics. <https://doi.org/10.3115/974557.974587>
- Wang, L [Liang], Sun, M., Zhao, W., Shen, K. & Liu, J. (2018). Yuanfudao at SemEval-2018 Task 11: Three-way Attention and Relational Knowledge for Commonsense Machine Comprehension. *Proceedings of the 12th International Workshop on Semantic Evaluation*, 758–762. <https://doi.org/10.18653/v1/S18-1120>
- Wang, X., Tsvetkov, Y., Ruder, S. & Neubig, G. (2021). Efficient Test Time Adapter Ensembling for Low-resource Language Varieties. *Findings of the Association for Computational Linguistics: EMNLP 2021*, 730–737. <https://doi.org/10.18653/v1/2021.findings-emnlp.63>
- Wei, C.-H., Harris, B. R., Kao, H.-Y. & Lu, Z. (2013). tmVar: a text mining approach for extracting sequence variants in biomedical literature. *Bioinformatics (Oxford, England)*, 29(11), 1433–1439. <https://doi.org/10.1093/bioinformatics/btt156>
- Wei, J. & Zou, K. (2019). EDA: Easy Data Augmentation Techniques for Boosting Performance on Text Classification Tasks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 6381–6387. <https://doi.org/10.18653/v1/D19-1670>
- Weights & Biases. (2025, 7. April). *Weights & Biases: The AI Developer Platform*. <https://wandb.ai/>
- Wilde, T. & Hess, T. (2007). Forschungsmethoden der Wirtschaftsinformatik. *WIRTSCHAFTSINFORMATIK*, 49(4), 280–287. <https://doi.org/10.1007/s11576-007-0064-z>
- Wirth, R. & Hipp, J. (2000). CRISP-DM: Towards a Standard Process Model for Data Mining. <https://www.semanticscholar.org/paper/CRISP-DM%3A-Towards-a-Standard-Process-Model-for-Data-Wirth-Hipp/48b9293cfd4297f855867ca278f7069abc6a9c24>
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., Platen, P. von, Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., . . . Rush, A. (2020). Transformers: State-of-

- the-Art Natural Language Processing. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 38–45.
<https://doi.org/10.18653/v1/2020.emnlp-demos.6>
- Wu, Z., Arora, A., Wang, Z., Geiger, A., Jurafsky, D [Dan], Manning, C. D. & Potts, C. (2024, 4. April). *ReFT: Representation Finetuning for Language Models*.
<http://arxiv.org/pdf/2404.03592>
- Yepes, A. J., MacKinlay, A., Gunn, N., Schieber, C., Faux, N., Downton, M., Goudey, B. & Martin, R. L. (2018). A hybrid approach for automated mutation annotation of the extended human mutation landscape in scientific literature. *AMIA ... Annual Symposium proceedings. AMIA Symposium, 2018*, 616–623. <https://pmc.ncbi.nlm.nih.gov/articles/PMC6371299/>
- Yepes, A. J. & Verspoor, K. (2014). Mutation extraction tools can be combined for robust recognition of genetic variants in the literature. *F1000Research*, 3, 18.
<https://doi.org/10.12688/f1000research.3-18.v2>
- Zhang, Y. & Zhang, H. (2022). *FinBERT-MRC: financial named entity recognition using BERT under the machine reading comprehension paradigm*.
<http://arxiv.org/pdf/2205.15485>
- Zhuang, F., Qi, Z., Duan, K., Xi, D., Zhu, Y., Zhu, H., Xiong, H. & He, Q. (2021). A Comprehensive Survey on Transfer Learning. *Proceedings of the IEEE*, 109(1), 43–76.
<https://doi.org/10.1109/JPROC.2020.3004555>
- Zhuang, L., Wayne, L., Ya, S. & Jun, Z. (2021). A Robustly Optimized BERT Pre-training Approach with Post-training. *Proceedings of the 20th Chinese National Conference on Computational Linguistics*, 1218–1227. <https://aclanthology.org/2021.ccl-1.108/>